2 **(a)** A program contains a global 1D array of type `STRING` containing 65 elements.

An existing text file is used to store the data in the array. Only non-blank elements (those that do **not** contain an empty string) are written to the text file.

An algorithm will:
- write the first element of the array as a new line in the text file
- continue until all elements have been written, each to a new line of the text file.

Stepwise refinement is applied to the algorithm.

Describe up to **six** steps for this algorithm that could be used to produce pseudocode.

Do **not** use pseudocode statements in your answer.

Step 1 .....

.....

Step 2 .....

.....

Step 3 .....

.....

Step 4 .....

.....

Step 5 .....

.....

Step 6 .....

.....

[6]

3 A program uses a stack to hold up to 30 integer values. The stack is implemented using a global integer variable and a global 1D array.

The array is declared in pseudocode:

```
DECLARE ThisStack : ARRAY[1:30] OF INTEGER
```

Stack design notes:

- The global variable `SP` acts as a stack pointer. `SP` contains the array index of the last value pushed onto the stack.
- If the stack is empty, then `SP` is assigned the value zero.
- The first item added to the stack will be stored in `ThisStack[1]`
- `SP` is incremented each time an item is added to the stack.

A function `Pop()` is written to remove an item from `ThisStack`. The function returns an item of type `PopData` which is defined in pseudocode:

```
TYPE PopData
   DECLARE Data   : INTEGER
   DECLARE Exists : BOOLEAN
ENDTYPE
```

The value removed from the stack is assigned to `Data` and `Exists` is set to `TRUE`. If it is **not** possible to remove a value from the stack, then `Exists` is set to `FALSE`

Complete the pseudocode for `Pop()`

```
FUNCTION Pop() RETURNS PopData

   DECLARE ThisPop : .....

   IF ..... THEN

      PopData.Exists ← ..... // Stack is empty

   ELSE

      PopData.Data ← .....

      PopData.Exists ← .....

      SP ← .....

   ENDIF

   RETURN ThisPop

ENDFUNCTION
```

[5]

4 A program contains a global 1D array `Data` containing 20 elements of type `INTEGER`

A global string `NumString` represents a sequence of three-digit numbers, separated by commas. For example:

```
"101,456,219,754,328"
```

The string contains at least **four** three-digit numbers. The total number of three-digit numbers in the string is unknown.

A procedure `Store()` will:

- extract one three-digit number at a time from `NumString`
- convert each of the three-digit numbers extracted to an integer and assign this to the next array element, starting from index 1
- end when all three-digit numbers have been stored, or when the array is full.

Complete the pseudocode for `Store()`

All local variables used must be declared.

```
PROCEDURE Store()
```

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

```
ENDPROCEDURE
```

[6]

5 A global 1D array `Num` of integers contains four elements. A program assigns values to these elements as shown:

```
Num[1] ← 1
Num[2] ← 2
Num[3] ← 5
Num[4] ← 3
```

A procedure `Process()` manipulates the values in the array.

The procedure is written in pseudocode:

```
PROCEDURE Process(Start : INTEGER)
   DECLARE CaseVar, Index, Count : INTEGER

   Index ← Start
   Count ← 0

   WHILE Count <= 20
      CaseVar ← Num[Index]
      CASE OF CaseVar
         1 : Num[Index] ← Num[Index] + Index //clause one
             Index ← Index + 1
             Count ← Count + 1

         2 : Num[Index] ← Num[Index] + Index //clause two
             Index ← Index + 2
             Count ← Count + 2

         3 : Num[Index] ← Num[Index] * 2     //clause three
             Index ← Index + 3
             Count ← Count - 1

         4 : Count ← Count + 4               //clause four
         OTHERWISE : Count ← 20
      ENDCASE

      Index ← (Index MOD 4) + 1

   ENDWHILE

ENDPROCEDURE
```

(a) Complete the trace table by dry running the procedure when it is called in the statement:
`CALL Process(1)`

Index	CaseVar	Count	Num[1]	Num[2]	Num[3]	Num[4]

[5]

8 A program is being developed to manage student book loans from a college library.

The programmer has defined a record type to define each loan.

The data items are:

Data item	Data type	Comment
`StudentID`	`STRING`	the unique ID of the student who has borrowed the book The first three characters of a `StudentID` represent a tutor ID. Each student has one tutor.
`BookID`	`STRING`	the unique ID of the book being borrowed
`OnLoan`	`BOOLEAN`	`TRUE` if the book has **not** been returned

The programmer has defined a global array `Loan` to store 8000 loan records.

There are more elements in the array than books in the library. Unused elements have the `StudentID` set to an empty string. These may occur anywhere in the array.

The programmer has defined a program module:

Module	Description
`LoanStatus()`	• called with two parameters of type `STRING` representing a `StudentID` and a `BookID` • outputs a message saying whether a given loan has been returned or not • outputs a warning message if a record of the given loan is **not** found

(a) Write efficient pseudocode for the module `LoanStatus()`

Assume that each combination of `StudentID` and `BookID` can occur only once.

[8]

(b) A second module is defined:

Module	Description
`LoansPerTutor()`	• called with a parameter of type `STRING` representing a tutor ID (as a reminder, the first three characters of a `StudentID` represent a tutor ID) • returns an integer value representing the number of books currently on loan to students who have the given tutor

Reminder: unused elements have the `StudentID` set to an empty string. These may occur anywhere in the array.

Write pseudocode for the module `LoansPerTutor()`

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....[7]

(c) It is decided to mark as unused all records for book loans that have been returned. The data for these records will first be written to a new text file for archive purposes.

(i) The archive program will automatically generate a meaningful filename each time it is run.

Outline a meaningful format to use for the filename.

.....

.....

.....

..... [2]

(ii) There is a problem that will need to be overcome before the data items can be written to a text file.

As a reminder, the data items are:

Data item	Data type	Comment
`StudentID`	`STRING`	the unique ID of the student who has borrowed the book The first three characters of a `StudentID` represent a tutor ID. Each student has one tutor.
`BookID`	`STRING`	the unique ID of the book being borrowed
`OnLoan`	`BOOLEAN`	`TRUE` if the book has **not** been returned

Explain the problem.

.....

..... [1]

(iii) One way of storing the data items in a text file is to store each data item on a separate line.

Identify **one** benefit and **one** drawback of this way of storing the data.

Benefit .....

.....

Drawback .....

.....

[2]

Refer to the **insert** for the list of pseudocode functions and operators.

1 (a) The table contains pseudocode examples.

Each example may contain statements that relate to one or more of:

- selection
- iteration (repetition)
- subroutines (procedures or functions).

Complete the table by placing **one or more** ticks (‘✓’) in each row.

Pseudocode example	Selection	Iteration	Subroutine

`IF Status = FALSE THEN`
`  FOR Count ← 1 TO 20`
`  CALL Reset(Count)`
`  NEXT Count`
`ENDIF`			

`OTHERWISE : Status ← TRUE`			
`WHILE AllDone() = TRUE`			
`30 : NextChar ← 'X'`			

[4]

(b) Complete the table by giving the appropriate data type.

Variable	Example data value	Data type
`Result`	`5.42`	
`MonthLetter`	`"JFMAMJJASOND"`	
`Birthday`	`15/11/2009`	

[3]

2 `Data` is a global 1D array containing 20 elements of type `REAL`

An algorithm will:

- input a sequence of real values, one at a time
- assign each value to consecutive array elements, starting from index 1
- end when the value 99.9 is input, or all 20 elements have been assigned (the value 99.9 must **not** be stored in the array).

Complete the program flowchart to represent the algorithm:



[5]

3 A text file `OldFile.txt` contains IDs, names and email addresses for members of a club. Three information items are stored for each member and each item is stored on a separate line.

The example shows the information for the first two members in the first six lines of the file:

Line in file	Information item	Example data
1	member 1 ID	`"AB1234"`
2	member 1 name	`"Freddie Jones"`
3	member 1 email address	`"FreddieJ909@Cambridge.org"`
4	member 2 ID	`"BC2345"`
5	member 2 name	`"Sue Smith"`
6	member 2 email address	`"Sue1024@Cambridge.org"`

The member ID string is always two letters followed by four digits.

The file design is to be changed so that information for each user is stored as a single line of the file, with the character `'\'` used as a separator between data items.

For example, the single line for member 1 will be:

```
"AB1234\Freddie Jones\FreddieJ909@Cambridge.org"
```

An algorithm will produce a new file `NewFile.txt` from the contents of `OldFile.txt`

Assume:

- `LineX`, `LineY` and `LineZ` are of type `STRING` and are used to store the three items of information for each member
- `NewString` is a temporary variable of type `STRING`
- `OldFile.txt` exists and contains valid data.

(a) The algorithm to create the new file is expressed in steps.

Complete the following numbered steps:

Step 1 : open the file ..... in .....

Step 2 : open the file ..... in .....

Step 3 : read a line from ..... and store in .....

Step 4 : read a line from ..... and store in .....

Step 5 : read a line from ..... and store in .....

Step 6 : set `NewString` to .....

Step 7 : write `NewString` to .....

Step 8 : repeat from step ..... until .....

Step 9 : close both files.

[6]

(b) The character `'\'` has been chosen as a separator.

Explain why this is a suitable character.

.....

..... [1]

(c) Two new information items are to be stored for each member. Both new items are encrypted; they can each contain any character (including `'\'`) and can be of any length up to a maximum of 99 characters.

For example:

Information item	Example data
member 1 ID	`"AB1234"`
member 1 name	`"Freddie Jones"`
member 1 email address	`"FreddieJ909@Cambridge.org"`
member 1 new information 1	`"En/98&*(|?\/D7iP"`
member 1 new information 2	`"23\CoboL"`

Explain the additional file design changes needed so that:

- all the information items for each member are stored on a single line of `NewFile.txt`
- it is possible to extract each information item after the line is read.

Assume the `'\'` character is not used in any email address.

.....

.....

.....

.....

.....

.....

.....

.....

..... [3]

4 A quiz has nine questions. There are:

- five easy questions each worth 3 points
- four hard questions each worth 5 points.

At the end of the quiz the points for each correctly answered question are added to give a total score.

A check is made to test that the total score is valid using a 1D array `CheckTotal` of type `BOOLEAN`. Each index value of the array represents a possible total score. The corresponding element value is `TRUE` if the index value represents a valid total score and `FALSE` otherwise.

The first nine rows of the array are:

Index value	Element value	Comment
0	`TRUE`	valid total score (no correct answers)
1	`FALSE`	invalid total score
2	`FALSE`	invalid total score
3	`TRUE`	valid total score (one 3-point question correct)
4	`FALSE`	invalid total score
5	`TRUE`	valid total score (one 5-point question correct)
6	`TRUE`	valid total score (two 3-point questions correct)
7	`FALSE`	invalid total score
8	`TRUE`	valid total score (one 3-point question and one 5-point question correct)

For example, a total score of 6 points is valid; the value of the array element at index value 6 is `TRUE`

(a) Write pseudocode to declare `CheckTotal` and to set all elements of the array to `FALSE`

All variables used must be declared.

.....

.....

.....

.....

.....

.....

.....

..... [4]

(b) The pseudocode represents an algorithm to set the appropriate elements of the array to `TRUE`

Complete the pseudocode:

```
DECLARE EasyQ, HardQ : INTEGER

FOR EasyQ ← ..... TO 15 STEP .....

   FOR HardQ ← 0 TO ..... STEP .....

      CheckTotal[.....] ← TRUE

   NEXT HardQ

NEXT EasyQ
```

[5]

8 A program is being developed to manage student book loans from a college library.

The programmer has defined a record type to define each loan.

The data items are:

Data item	Data type	Comment
`StudentID`	`STRING`	the unique ID of the student who has borrowed the book
`BookID`	`STRING`	the unique ID of the book being borrowed
`OnLoan`	`BOOLEAN`	`TRUE` if the book has **not** been returned

The programmer has defined a global array `Loan` to store 7000 loan records.

There are more elements in the array than books in the library. Unused elements have the `StudentID` set to an empty string. These may occur anywhere in the array.

The programmer has defined a program module:

Module	Description
`CountLoans()`	• called with a parameter of type `STRING` representing a `StudentID` • counts the number of books currently on loan to the specified student • counts the number of books that the student has already returned • output both counts together with a suitable message

(a) Write pseudocode for module `CountLoans()`

.....

.....

.....

(b) As a reminder, a global array `Loan` stores 7000 loan records with data items for each loan record:

Data item	Data type	Comment
`StudentID`	`STRING`	the unique ID of the student who has borrowed the book
`BookID`	`STRING`	the unique ID of the book being borrowed
`OnLoan`	`BOOLEAN`	`TRUE` if the book has **not** been returned

A new module is defined:

Module	Description
`NewLoan()`	• called with two parameters of type `STRING` representing a `StudentID` and a `BookID` • searches the array for an unused loan record • if found, updates the loan record and returns `TRUE`, otherwise returns `FALSE`

Write efficient pseudocode for `NewLoan()`

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [7]

(c) When a book is returned, the loan record will have its `OnLoan` data set to `FALSE`

A new procedure `Archive()` will mark these records as unused after first saving them for future reference.

Explain how these records can be saved for future reference.

.....

.....

.....

..... [2]

(d) It is decided to extend the program so that a new module `Reminder()` will send an email to the student three days before the book is due to be returned.

Outline the changes that will need to be made to the data stored and how this data will be used to generate the reminder email.

Data .....

.....

.....

.....

Use .....

.....

.....

.....

[2]

2 `Data` is a global 1D array containing 30 elements of type `STRING`

An algorithm will output:

- all non-blank elements (elements that do **not** contain an empty string)
- the final total of the number of elements output.

Complete the program flowchart to represent the algorithm:





[5]

3 A program is needed to manage individual rentals in a car-hire business.

The data items for each rental will be held in a record structure of type `RentalRecord`
The programmer has started to define the items that will be needed:

Item	Example value	Comment
`RentalID`	`"AB1234"`	a unique alpha-numeric value
`CarID`	`241`	a numeric value used as an array index
`DisCode`	`'S'`	a letter indicating the type of discount offered
`Start`	`13/06/2025`	when the rental starts
`Duration`	`7`	the number of days of the rental
`Completed`	`FALSE`	`TRUE` when the car is returned and the rental charge paid

(a) (i) Write pseudocode to declare the record structure for type `RentalRecord`

.....

.....

.....

.....

.....

.....

.....

..... [4]

(ii) A 1D array `Rental` containing 500 elements is used to store the data for all rental records.

Write pseudocode to declare the `Rental` array.

.....

..... [2]

(b) State **three** benefits of using an array of records to store the data for all rentals.

1 .....

.....

2 .....

.....

3 .....

.....

4 A program contains a global 1D array `Number` consisting of 20 elements of type `REAL`

A procedure `Store()` will input a sequence of up to 20 real values, one value at a time. These values will be assigned to elements of the array using four steps:

Step 1: store the first value in the sequence in the first element of the array
Step 2: check each subsequent value input. If this value is larger than the previous value input, then assign the value to the next array element, otherwise go to **step 4**
Step 3: repeat from **step 2** unless the array is full
Step 4: output the count of the number of values stored in the array together with a suitable message.

(a) Complete the pseudocode for `Store()`

All variables used in the algorithm must be declared.

```
PROCEDURE Store()
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
ENDPROCEDURE
```

[6]

(b) The requirements of the program change:

- the number of values in the sequence is unknown, but may be higher than 20
- the values will need to be accessed by another program as data.

The data will be stored in a text file instead of an array.

(i) Give **two** benefits of using a text file instead of an array.

1 .....

.....

2 .....

.....

[2]

(ii) State any change that will need to be made before each value is written to the file.

.....

..... [1]

5 A program is being designed in pseudocode.

The program contains the following declaration for the global array `MyData`:

```
DECLARE MyData : ARRAY[1:10000] OF STRING
```

A function `FindFirst()` is written to search the array for a given string and to return the index of the first element where that string is found, or to return –1 if the string is **not** found.

The function is written in pseudocode as shown:

```
FUNCTION FindFirst(SearchString : STRING) RETURNS INTEGER
   DECLARE Index, FoundAt : INTEGER
   FoundAt ← -1
   FOR Index ← 1 TO 10000
      IF MyData[Index] = SearchString THEN // outer conditional clause
         IF FoundAt = -1 THEN              // inner conditional clause
            FoundAt ← Index
         ENDIF
      ENDIF
   NEXT Index
   RETURN FoundAt
ENDFUNCTION
```

(a) (i) Comment on why the programmer chose –1 as the initial value of `FoundAt`

.....

..... [1]

(ii) Explain the purpose of the **outer** conditional clause.

.....

.....

..... [1]

(iii) The **inner** conditional clause ensures that only the index of the **first** matching element, if any, is returned.

Explain how this clause works.

.....

.....

..... [1]

8 A program is being developed to manage student book loans from a college library. Students may borrow up to five books at a time from the library.

The programmer has defined a record type to define each loan.

The record data items are:

Data item	Data type	Comment
`StudentID`	`STRING`	the unique ID of the student who has borrowed the book
`BookID`	`STRING`	the unique ID of the book being borrowed
`OnLoan`	`BOOLEAN`	`TRUE` if the book has **not** been returned

The programmer has defined a global array `Loan` to store 5000 loan records.

There are more elements in the array than books in the library. Unused elements have the `StudentID` set to an empty string. These may occur anywhere in the array.

The programmer has defined the first program module:

Module	Description
`OKToBorrow()`	• called with a parameter of type `STRING` representing a `StudentID` • search the array for loan records for the specified student • output a suitable message to say whether the student may, or may not, borrow another book

(a) Write efficient pseudocode for the module `OKToBorrow()`

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [7]

(b) A second module is defined:

Module	Description
`ReturnBook()`	• called with two parameters of type `STRING` representing a `StudentID` and a `BookID` • searches the array for the relevant loan record • when found, sets `OnLoan` to `FALSE` and returns `TRUE` • if a loan record is **not** found, or the book has already been returned, then returns `FALSE`

Write efficient pseudocode for the module `ReturnBook()`

Assume that each student is only allowed to borrow each book only once. That means that there will be no more than **one** loan record for a given combination of student and book.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [8]

(c) It is decided to introduce a system of fines for books that have been borrowed for too long.

Two new requirements are defined:

1. Each loan has a maximum length, represented as a number of days.
2. Each book in the library will be assigned one of three categories. Each category has a different maximum loan length.

Record structure and program design changes are needed to meet these two requirements.

Outline the changes that are necessary to meet the **two** requirements.

.....

.....

.....

.....

.....

.....

..... [2]

Refer to the **insert** for the list of pseudocode functions and operators.

1 A program is being developed for the management of a sports centre.

(a) The programmer developing the software decides to use modules (procedures or functions) and local variables. They also decide not to use global variables.

(i) State **two** reasons why the programmer decides to use modules.

1 .....

.....

2 .....

.....

[2]

(c) The table lists some of the variables used in the program.

Complete the table by writing the most appropriate data type for each variable.

Variable	Use of variable	Data type
`MemberCount`	stores how many members have used the sports centre each day	
`TotalTakings`	stores the total amount of money taken each day	
`BookingConfirmed`	stores the state of a booking for an exercise class; a booking is either confirmed or **not** confirmed	
`MemberDOB`	to calculate when to send an email with a birthday message to a member	

[4]

2 A programmer uses a number of Abstract Data Types (ADT) in the programs that she is developing.

(a) A stack using memory locations 400 to 409 is used in one program.

The diagram represents the current state of the stack.

A variable `TopOfStack` pointer indicates the last value added to the stack.

Stack

Memory location	Value	
409		
408		
407		
406		
405		
404		
403	'P'	← `TopOfStack`
402	'X'	
401	'D'	
400	'B'	

(i) Complete the answer column in the following table:

	Answer
the memory location of the value that has been on the stack for the longest time	
the number of consecutive **push** operations that will result in the `TopOfStack` variable containing 409	

[2]

(ii) The diagram shows the current state of the stack.

Stack

Memory location	Value	
409		
408		
407	'A'	← TopOfStack
406	'C'	
405	'F'	
404	'K'	
403	'B'	
402	'S'	
401	'R'	
400	'D'	

The following sequence of operations are performed:

```
PUSH 'T'
POP
POP
PUSH 'Z'
PUSH 'X'
POP
PUSH 'Y'
```

Complete the following diagram to show the state of the stack after the operations have been performed.

Stack

Memory location	Value
409	
408	
407	
406	
405	
404	
403	
402	
401	
400	

[3]

(b) In a different program, the programmer decides to implement a linked list using an array of records.

(i) The array is represented in the diagram.

A pointer value of `-1` indicates the end of the linked list.

Complete the pointer field to produce a linked list that is in alphabetical order.

Index	Data	Pointer
0	"Neptune"	
1	"Jupiter"	
2	"Saturn"	
3	"Earth"	
4	"Mercury"	
5	"Uranus"	

[3]

(ii) The variable `StartPointer` contains the index of the first item in the linked list.

State the value of the variable `StartPointer`

..... [1]

(c) A third program is also being created to manage a queue.

Describe **two** features of a queue.

Feature one .....

.....

Feature two .....

.....

[2]

4 A programmer is developing a new stock control program for a shop owner to produce sales reports.

(a) A text file `Sales.txt` stores strings representing the number of items sold.

The file contains 52 lines, each representing the number of items sold in each week of the year.

For example:

File line number	File data
1	"350"
2	"502"
3	"434"
49	"492"
50	"872"
51	"772"
52	"201"

The example shows that 502 items were sold in week 2.

The owner requires a module to output all week numbers where the number of items sold is greater than 500.

Describe an algorithm that produces the required module.

Do **not** include pseudocode statements in your answer.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [5]

7 A program is being developed to implement a customer loyalty scheme for a coffee shop.

The programmer has decided that the following data items need to be stored for each customer:

Data item	Description
customer ID	a unique six-digit string
loyalty points	an integer value that depends on how often the customer visits the coffee shop
last visit date	the date the customer last visited the coffee shop

The programmer has defined a program module:

Module	Description
`UpdateVisit()`	• called with two parameters: 1. loyalty points of type `INTEGER` 2. last visit date of type `DATE` • change the loyalty points: ○ increase by **four** if the last visit date and the current date are in the same month ○ otherwise increase by **one** • change the last visit date to the current date • the changed values must be available to the code that follows the call to `UpdateVisit()`

(a) (i) Write pseudocode for module `UpdateVisit()`

Assume the customer has made at least **one** previous visit.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.... [5]

(b) A text file `Loyalty.txt` will be used to store the data items for the loyalty scheme.

The text file `Loyalty.txt` contains only one line of data for each customer.

Each data item is separated by a comma:

`<CustomerID>,<LoyaltyPointsString>,<LastVisitDateString>`

`LastVisitDateString` is always eight characters in length in the format: DDMMYYYY.

An example of how a line of data will be stored in the text file `Loyalty.txt` is:

```
"100123,132,05032025"
```

This example shows that the customer with an ID of 100123 had 132 loyalty points following their last visit to the coffee shop on 05/03/2025.

If a customer visits the coffee shop on a Monday, the value of their loyalty points is increased by 10.

The programmer has defined **two** more program modules:

Module	Description
`FindCustomer()` (is already written)	• called with a parameter of type `STRING` that represents a customer ID • returns the line of data read from the text file `Loyalty.txt` containing the data items for that customer
`MondayCheck()`	• called with a parameter of type `STRING` that represents the customer ID of a customer • calls `FindCustomer()` • extracts the loyalty points from the string returned by `FindCustomer()` • increases the loyalty points for the current customer by 10 if the day visited is a Monday • returns an integer value representing the loyalty points

(i) Write pseudocode for module `MondayCheck()`

The module `FindCustomer()` must be used and assume it returns a valid string for the current customer.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [8]

(ii) A date is stored in `LastVisitDateString` in the format:

DDMMYYYY

DD is a 2-digit string
MM is a 2-digit string
YYYY is a 4-digit string.

For example, the date 03/09/2024 is stored as `"03092024"`

An algorithm is needed to convert the string stored in `LastVisitDateString` to data type `DATE` which is then stored in the variable `LastVisitDate`.

Complete the pseudocode for this algorithm:

Assume that `LastVisitDateString` has been declared and contains valid data.

```
DECLARE DayInt, MonthInt, YearInt : INTEGER

DECLARE LastVisitDate : DATE
```

.....

.....

.....

.....

.....

.....

.....

.....

[4]

Refer to the **insert** for the list of pseudocode functions and operators.

1 A program is being developed to control the production line in a factory.

(a) A number of different program life cycles are available for the development of a program.

(i) Explain the need for different program development life cycles.

.....

..... [1]

(c) Pseudocode has been used to design modules for the program to control the production line.

The table shows **four** valid pseudocode expressions.

Complete the table by giving the data type of the evaluated expression.

Expression	Data type
`RIGHT(MachineCode, 4)`	
`Speed * 2.5`	
`NOT Status`	
`IS_NUM(Check)`	

[4]

(d) A global array `Product` is used as part of the pseudocode design being developed to control the production line. `Product` is used to store the number of rejected items each day.

The following pseudocode statement is used to assign a value to an element of the array:

```
Product[x, y] ← 23
```

The lower and upper bound values are shown in the table:

Variable	Lower bound	Upper bound
x	0	99
y	0	9

(i) State the number of dimensions of `Product`

..... [1]

(ii) State the total number of elements in array `Product`

..... [1]

(iii) Give the pseudocode declaration for the array `Product`

..... [2]

4 An algorithm will:

- input 100 integer values, one value at a time
- store the first value input into the first location of the array `Number`
- store the next input value in the next unused location of the array `Number`
- output the contents of `Number` array in the opposite sequence to that in which the values were input.

Complete the program flowchart to represent the algorithm.

Variable declarations are **not** required.

START

END

[6]

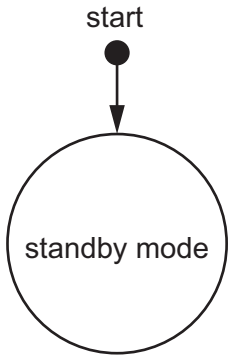
5 An automated digital camera system is used to take a sequence of pictures of animals in a garden. During the design of the system, a state-transition diagram is produced.

The table details the states in the automated digital camera system along with the events which cause the states to change.

The system starts in standby mode. The sequence of pictures is taken when in active mode.

Current state	Event	Next state
standby mode	turn on	detect mode
detect mode	turn off	standby mode
detect mode	movement detected	active mode
active mode	20 seconds elapsed	sequence complete
active mode	turn off	standby mode
sequence complete	time saved	detect mode

(a) Complete the state-transition diagram for the automated digital camera system.



[4]

(b) At the end of each sequence of pictures, the time is saved as a string in the format `<HH><MM><SS>` where:

- `HH` represents the hours using two digits
- `MM` represents the minutes using two digits
- `SS` represents the seconds using two digits.

For example:

- `"081230"` is stored to represent the time 8:12:30, in the morning
- `"152235"` is stored to represent the time 15:22:35, in the afternoon.

Each string is stored on a new line in the text file `TimeTaken.txt`

An algorithm is required to process the content of the text file `TimeTaken.txt` once it has been transferred to the computer.

For each hour when pictures are taken, output a suitable message showing the hour and the total number of sequences of pictures taken within that hour.

Example outputs:

```
Hour : 15  Total : 32
Hour : 18  Total : 1
```

Write pseudocode for this algorithm.

Assume the text file `TimeTaken.txt` contains at least **one** line.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [8]

6 Two arrays `Data` and `Pointer` are accessed by the procedure `Place()`

`Data` and `Pointer` are both global arrays of type `INTEGER`

The contents of these two arrays are shown:

	Data
1	1018
2	1007
3	1010
4	1056
5	1092
6	1062
7	1034
8	0
9	0
10	0

	Pointer
1	7
2	3
3	1
4	6
5	-1
6	5
7	4
8	9
9	10
10	-1

Study the pseudocode:

```
PROCEDURE Place(Value : INTEGER, Start : INTEGER, Unused : INTEGER)
   DECLARE New, Current, Last : INTEGER
   CONSTANT NullPointer = -1
   New ← Unused
   Last ← NullPointer
   Current ← Start
   WHILE Current <> NullPointer AND Data[Current] < Value
      Last ← Current
      Current ← Pointer[Current]
   ENDWHILE
   Pointer[New] ← Pointer[Last]
   Pointer[Last] ← New
   Data[New] ← Value
ENDPROCEDURE
```

(a) **(i)** Complete the trace table below by dry running the procedure `Place()` when it is called by the statement:

```
CALL Place(1043, 2, 8)
```

Value	Start	Unused	New	Last	Current

[4]

(ii) Complete the diagram showing the contents of the global arrays `Data` and `Pointer` after the procedure `Place()` has run to completion when called as shown in part **(a)(i)**.

	Data
1	1018
2	1007
3	1010
4	1056
5	1092
6	1062
7	1034
8	
9	
10	

	Pointer
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	

[3]

(b) The operation carried out by procedure `Place()` together with the arrays form part of the implementation of an Abstract Data Type (ADT).

Identify the ADT and state the operation carried out by procedure `Place()`

.....

.....

.....

..... [2]

7 A program is being developed to implement a customer loyalty scheme for a coffee shop.

Each customer has a unique customer ID starting at `10001` with this value increasing by one each time a new customer joins the loyalty scheme.

For example, the third customer who joins the loyalty scheme is given the customer ID `10003`

The loyalty scheme is limited to 1000 customers.

A customer is awarded a loyalty point every time they buy a coffee.

The programmer has decided to use a global 2D array `Loyalty` of type `INTEGER`. The array `Loyalty` is made up of 1000 rows and 2 columns. Each row relates to one customer; column 1 contains the unique customer ID and column 2 contains the number of customer loyalty points.

Rows in the array `Loyalty` that are not currently being used have the value of Column 1 set to `99999`

The array is sorted in ascending order by customer ID.

The programmer has defined a program module:

Module	Description
`FindCustomer()`	• called with parameter of type `INTEGER` representing a customer ID • searches the `Loyalty` array for this customer ID • the search will stop as soon as the customer ID is found • the search should efficiently deal with the situation when the customer ID is **not** stored in the `Loyalty` array • if the customer ID is found, return an integer value representing the loyalty points, otherwise return `-1`

(a) Write efficient pseudocode for module `FindCustomer()`

[8]

(b) A customer can claim a free coffee for every 11 loyalty points.

The programmer has defined a second program module:

Module	Description
`PointsReport()`	• output the customer ID for each customer who has 11 or more loyalty points • output the average loyalty points for all customers in the `Loyalty` array along with an appropriate message

Write efficient pseudocode for module `PointsReport()`

Assume the array contains the data for at least **one** customer.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [7]

(c) The programmer decides to amend the customer ID; it will be stored as a `STRING` instead of an `INTEGER`. This means that the 2D array `Loyalty` can no longer be used.

(i) Explain why the 2D array `Loyalty` can no longer be used.

.....

..... [1]

(ii) Explain how a 1D array could be used to store both the loyalty points and the amended customer ID.

.....

.....

.....

..... [2]

Refer to the **insert** for the list of pseudocode functions and operators.

1 A program is being developed to help the manager of a shop control the stock.

(a) An identifier table has been used during the design stage.

Complete the identifier table:

Example value	Explanation	Variable name	Data type
`"Fruit"`	a category of stock that is sold in the shop		
`20/02/2025`	when an item was sold		
`12.67`	the cost of an item		
`TRUE`	to indicate if an item is in stock		

[4]

4 Study the algorithm:

```
DECLARE Chars : ARRAY[1:4] OF CHAR
DECLARE I, J : INTEGER
DECLARE Key : CHAR
I ← 2
Chars[1] ← 'D'
Chars[2] ← 'T'
Chars[3] ← 'H'
Chars[4] ← 'R'
WHILE I <= 4                          //Outer loop
   Key ← Chars[I]
   J ← I - 1
   WHILE J >= 0 AND Chars[J] > Key     //Inner loop
      Chars[J + 1] ← Chars[J]
      J ← J - 1
   ENDWHILE
   Chars[J + 1] ← Key
   I ← I + 1
ENDWHILE
```

(a) The outer loop structure used in the algorithm is **not** the most appropriate one to use.

State the type of loop structure that would be the most appropriate to use and justify why it is the most appropriate.

Loop structure .....

Justification .....

.....

[2]

(b) Complete the trace table by dry running the algorithm.

The first row has been completed.

I	Key	J	Chars[J]	Chars			
				[1]	[2]	[3]	[4]
2				'D'	'T'	'H'	'R'

[6]

5 Stacks and queues are both abstract data types.

A stack uses a top-of-stack pointer to indicate the location of the last item added to the stack.

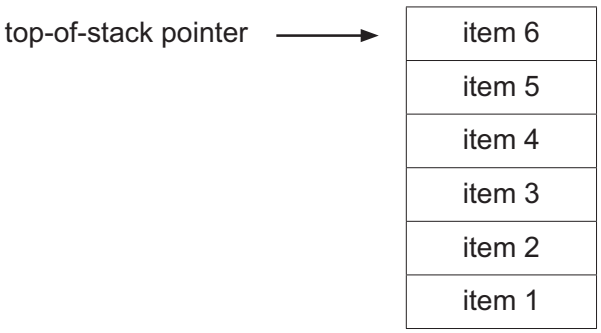
A queue uses two pointers:

- a front pointer to indicate the location of the next item to be removed from the queue
- a rear pointer to indicate the location of the next item to be added to the queue.

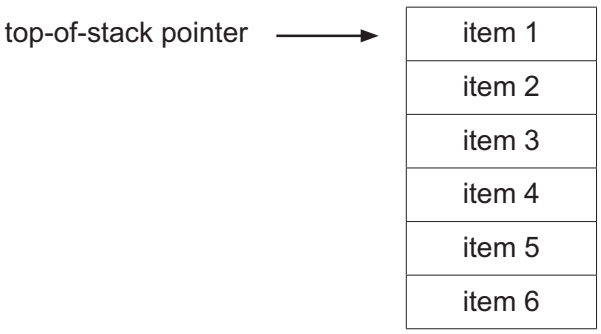
A queue can be used to reverse the items stored on a stack.

For example, if a stack contains six items:

Initial state of the stack:



Final state of the stack when the items have been reversed:



Describe how the queue could be used to reverse the items that are currently stored on the stack.

Your description must include how the pointers are used in both the stack and queue.

Assume:

- The stack initially contains an unknown number of items.
- The queue can store all the items currently stored on the stack.
- The queue is initially empty.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [7]

6 A program monitors the speed of vehicles as they move around a large building site.

Each vehicle contains a sensor which reads an integer value that represents the speed of the vehicle. The value is expected to be in the range 0 to 60 inclusive.

The sensors cannot read values less than 0.

A program module has been written to validate the values read by the sensors.

(a) A test plan is needed to fully test the module.

Complete the table. The first line has been completed for you.

Assume the sensors generate only integer values.

Type of test data	Test data value	Expected outcome
normal	36	data item is accepted

[4]

(b) Each sensor used in the system has a unique sensor ID number in the range 1 to 50.

The program that is used to monitor the speed of each vehicle reads each sensor value every second and stores this value along with the sensor's unique ID into a global 2D array `Reading`

The global array `Reading` has been declared as follows:

```
DECLARE Reading : ARRAY[1:2000, 1:2] OF INTEGER
```

The array contains 4000 elements organised as 2000 rows and 2 columns.

Column 1 contains the sensor value, and column 2 contains the sensor ID.

When 2000 sensor readings have been taken, the array is full and the system stops taking any more sensor readings until the array has been processed.

A procedure `Sort()` is needed to sort the array into ascending order of sensor value using an efficient bubble sort algorithm.

Write **efficient** pseudocode for the procedure `Sort()`

[8]

7 A program is being developed to implement a customer loyalty scheme for a coffee shop.

The programmer has decided that the following data items need to be stored for each customer:

Data item	Description
customer ID	a unique six-digit string
points	an integer value that is increased by one for every cup of coffee the customer orders

When a customer visits the shop and orders coffee, the scheme operates as follows:

- The total number of points is increased by the number of coffees ordered.
- If just one cup of coffee is ordered and the number of points goes above 10, then:
  - the cup of coffee they have just ordered is given to them free of charge
  - the number of points is reduced by 11.
- If the order is for multiple coffees and the number of points goes above 10, then:
  - they get one coffee free of charge for every 11 points
  - the number of points is reduced by 11 for each free coffee.

For example, the:

- customer currently has 9 points
- customer orders 16 cups of coffee
- total number of points now becomes 25
- customer gets 2 free coffees and now has 3 points left.

The programmer has defined a program module that is called every time a customer places an order:

Module	Description
`CustomerOrder()`	• called with two integer parameters: ◦ the number of coffees ordered ◦ the current number of points • output a suitable message giving the number of free coffees • return a value for the new points total.

(a) Write pseudocode for module `CustomerOrder()`

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

(b) A text file `Loyalty.txt` will be used to store the data items for the loyalty scheme. The data items for each customer will be stored on a separate line of the text file where each data item is separated by a comma:

```
<CustomerID>,<Points>
```

The contents of the text file `Loyalty.txt` will always be stored in ascending order by customer ID.

When the data items are read from or written to the text file `Loyalty.txt`, they may need to be converted to the appropriate data type.

Each customer has a unique customer ID starting at `"100001"` with this value increasing by one each time a new customer joins the loyalty scheme.

When a customer joins the loyalty scheme, they are assigned the next customer ID and value of points is set to 0.

For example, if the loyalty scheme has 204 customers and a new customer joins the loyalty scheme, the following line is added to the text file `Loyalty.txt`:

```
"100205,0"
```

You can assume that the number of customers in the loyalty scheme will never be more than 9000.

The programmer has defined a program module as follows:

Module	Description
`AddNewCustomers()`	• called with an integer parameter representing the number of new customers to be added to the loyalty scheme • adds a new line, containing the required information, to the text file `Loyalty.txt` for each customer added to the loyalty scheme • outputs each new customer ID added to the loyalty scheme

(i) Write pseudocode for module `AddNewCustomers()`

Assume that there is at least **one** customer already in the loyalty scheme.

.....

.....

.....

.....

.....

.....

.....

....

.....

.....

.....

(ii) The requirements for `AddNewCustomers()` are changed. There may be no existing customers in the loyalty scheme.

Explain the changes that will need to be made to the module `AddNewCustomers()`

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [4]

1 **(a)** The following table contains pseudocode examples.

Each example may contain statements that relate to one or more of the following:

- selection
- iteration (repetition)
- subroutine (procedure or function).

Complete the table by placing **one or more** ticks ('✓') in each row.

Pseudocode example	Selection	Iteration	Subroutine

`FOR Index ← 1 TO 3`
`  IF Safe[Index] = TRUE THEN`
`  Flap[Index] ← 0`
`  ENDIF`
`NEXT Index`			

`CASE OF Compound(3)`			
`REPEAT UNTIL AllDone() = TRUE`			
`WHILE Result[3] <> FALSE`			

[4]

(b) Complete the table by giving the appropriate data type in each case.

Variable	Example data value	Data type
`Available`	`TRUE`	
`Received`	`"18/04/2021"`	
`Index`	`100`	

[3]

3 The implementation of a linked list uses an integer variable and a 1D array `List` of type `Node`.

Record type `Node` is declared in pseudocode as follows:

```
TYPE Node
   DECLARE Data : STRING
   DECLARE Pointer : INTEGER
ENDTYPE
```

The array `List` is declared in pseudocode as follows:

```
DECLARE List : ARRAY[1:200] OF Node
```

The linked list will operate as follows:

- Integer variable `HeadPointer` will store the array index for the first node in the linked list.
- The `Pointer` field of a node contains the index value of the next array element (the next node) in the linked list.
- The value 0 is used as a null pointer.

(a) (i) State why the value 0 has been selected as the null pointer.

.....

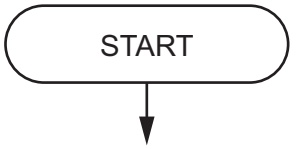
..... [1]

(ii) Give the range of valid values that could be assigned to variable `HeadPointer`.

..... [1]

(b) The array `List` will be initialised so that each node points to the following node. The last node will contain a null pointer.

Complete the program flowchart to represent the algorithm for this operation.



END

[4]

(c) An algorithm outputs the `Data` field from all nodes in the array `List`. The order the `Data` is output should be the same order it is stored in the linked list.

Describe the algorithm in **four** steps.

Do **not** use pseudocode statements in your answer.

Step 1 .....

.....

.....

Step 2 .....

.....

.....

Step 3 .....

.....

.....

Step 4 .....

.....

.....

[4]

BLANK PAGE

4 An examination paper has a maximum of 75 marks. One of five pass grades (A to E) is assigned, depending on the mark obtained. The lowest mark for a given grade is known as the grade boundary.

A program is being written to process examination marks.

The five grade boundaries are stored in a global 1D array `GB` of type `INTEGER`, for example:

Index	Value	Comment
1	65	The minimum mark for an A grade.
2	57	The minimum mark for a B grade.
3	43	The minimum mark for a C grade.
4	35	The minimum mark for a D grade.
5	27	The minimum mark for an E grade.

Any paper that achieves a mark within 2 marks of a grade boundary must be checked. Using the given table, a paper with 45 marks would need to be checked.

(a) The pseudocode algorithm to determine whether a paper should be checked is as shown. The mark for the paper is stored in variable `Mark`. Global variables `Mark`, `Index`, `Upper` and `Lower` are declared as integers.

Complete the pseudocode.

```
FOR Index ← 1 TO .....
   Lower ← GB[Index] - 2
   Upper ← .....
   IF Mark ..... AND Mark ..... THEN
      OUTPUT "Check this paper"
   ENDIF
NEXT Index
```

[4]

(b) An alternative algorithm to determine if a paper needs to be checked uses a global 1D array `Check`, containing 76 elements of type `BOOLEAN`. The indices of the array are from 0 to 75 (inclusive), corresponding to the range of possible marks.

An element value in `Check` is `TRUE` if the index is within 2 marks of a grade boundary. For example, in the case where the C grade boundary is 43 the corresponding part of the `Check` array would be as follows:

Index	Value	
40	FALSE	
41	TRUE	
42	TRUE	
43	TRUE	← The grade boundary for a C grade
44	TRUE	
45	TRUE	
46	FALSE	

(i) The mark for a given paper is stored in variable `Mark`.

Describe how an algorithm would use the `Check` array to determine whether this paper should be checked.

.....

.....

.....

..... [2]

(ii) A procedure `GBInitialise()` will initialise the `Check` array using values from the `GB` array.

Note it can be assumed that the maximum grade boundary value for A is 70 and the minimum value for E is 15.

Write pseudocode for the procedure.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

BLANK PAGE

8 A program is being developed to implement a game for up to six players.

During the game, each player assembles a team of characters. At the start of the game there are 45 characters available.

Each character has four attributes, as follows:

Attribute	Examples	Comment
Player	0, 1, 3	The player the character is assigned to.
Role	Builder, Teacher, Doctor	The job that the character will perform in the game.
Name	Bill, Lee, Farah, Mo	The name of the character. Several characters may perform the same role, but they will each have a unique name.
Skill level	14, 23, 76	An integer in the range 0 to 100, inclusive.

The programmer has defined a record type to define each character. The record type definition is shown in pseudocode as follows:

```
TYPE CharacterType
   DECLARE Player : INTEGER
   DECLARE Role : STRING
   DECLARE Name : STRING
   DECLARE SkillLevel : INTEGER
ENDTYPE
```

The `Player` field indicates the player to which the character is assigned (1 to 6). This field value is 0 if the character is **not** assigned to any player.

The programmer has defined a global array to store the character data, as follows:

```
DECLARE Character : ARRAY[1:45] OF CharacterType
```

At the start of the game all record fields are initialised, and all `Player` fields are set to 0

The programmer has defined a program module as follows:

Module	Description

`Count()`	• called with two parameters:
	  ○ an integer representing a player
	  ○ a string representing a character role
	• searches the `Character` array for characters with the given role that are assigned to the given player
	• counts the number of assigned characters and sums their total skill level
	• outputs the result of the search if characters with the given role are found, for example:
	`"Player 3 has 4 characters with the role of Teacher and the total skill level is 65"`
	• if no characters with the given role are found, outputs:
	`"No characters with that role are assigned to this player"`

(a) Complete the pseudocode for module `Count()`.

```
PROCEDURE Count(ThisPlayer : INTEGER, ThisRole : STRING)
```

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

```
ENDPROCEDURE
```

[7]

(b) The `Character` array data has been saved in the text file `SaveFile.txt`
Each line of the file contains one element of the array (one record).

New modules are defined:

Module	Description
`Extract()` (already written)	• called with two parameters: ○ a string representing a complete line from the text file ○ an integer representing a field number (see structure below) • returns a string representing the required field
`Restore()`	• opens the text file `SaveFile.txt` • reads lines from the file and assigns values to each record in the `Character` array using data from each line of the file

As a reminder, the record structure is repeated here:

```
TYPE CharacterType
   DECLARE Player : INTEGER       //Field number 1
   DECLARE Role : STRING          //Field number 2
   DECLARE Name : STRING          //Field number 3
   DECLARE SkillLevel : INTEGER   //Field number 4
ENDTYPE
```

Write pseudocode for module `Restore()`.

You must use the module `Extract()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [7]

(c) The game can last for several days and users often find that they have to close and rerun the game program many times in order to complete it.

Describe the benefit of using the file `SaveFile.txt` as described in part **(b)**.

.....

.....

.....

..... [2]

Refer to the **insert** for the list of pseudocode functions and operators.

1 A program has been developed and released for general use. After a few months of use an error is detected where under certain circumstances it outputs an unexpected value.

(a) The error in the program needs to be corrected.

Identify the stage of the program development life cycle that this correction is made in.

..... [1]

(c) To solve the error a programmer decides to create a new module.

The design of the new module has been completed and the module is being coded.

Identify **two** features of an IDE that will help during the coding of this new module.

1 .....

2 .....
[2]

(d) The new module referred to in part **(c)** introduces **three** new variables.

Complete the following table by giving the appropriate data type for each.

Variable name	Used to store	Data type
`Name`	A customer name.	
`Index`	An array index.	
`Result`	The result of the division of any two non-zero numbers.	

[3]

BLANK PAGE

2 A program is being developed to process bank card information.

When a card number is displayed, all the characters **except** the last four are replaced with the asterisk character `'*'`.

Card numbers are stored as strings. The strings are between 10 and 20 characters in length.

The function `Conceal()` will take a string representing a card number and return a modified string.

Example strings:

Original string	Modified string
`"1234567890"`	`"******7890"`
`"1234567897652"`	`"*********7652"`
`"1234567890123456"`	`"************3456"`

(a) The function `Conceal()` will:

- take a numeric string as a parameter representing the card number
- return a string in which the asterisk character replaces all except the last four characters of the card number parameter.

Write pseudocode for the function `Conceal()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

(b) The requirements have been changed. `Conceal()` will now be written as a procedure which will process 100 card numbers each time it is called.

The card numbers will be stored in a 2D array `CardNumber`. The original string will be stored in column one and the modified string in column two.

(i) Write pseudocode to declare the array.

.....

..... [2]

3 A program uses a stack to hold up to 60 numeric values.
The stack is implemented using two integer variables and a 1D array.

The array is declared in pseudocode as shown:

```
DECLARE ThisStack : ARRAY[1:60] OF REAL
```

The stack operates as follows:

- Global variable `SP` acts as a stack pointer that points to the next available stack location. The value of `SP` represents an array index.
- Global variable `OnStack` represents the number of values currently on the stack.
- The stack grows upwards from array element index 1.

(a) (i) Give the initial values that should be assigned to the **two** variables.

`SP` .....

`OnStack` .....
[1]

(ii) Explain why it is **not** necessary to initialise the array elements before the stack is used.

.....

.....

.....

..... [2]

(b) A function to add a value to `ThisStack` is expressed in pseudocode as shown.
The function will return a value to indicate whether the operation was successful or not.

Complete the pseudocode by filling in the gaps.

```
FUNCTION Push(ThisValue : REAL) RETURNS BOOLEAN

   DECLARE ReturnValue : BOOLEAN

   IF ..... THEN

      RETURN ..... // stack is already full

   ENDIF

   ..... ← ThisValue

   SP ← .....

   OnStack ← OnStack + 1

   RETURN TRUE

ENDFUNCTION
```

[4]

6 A shop sells sandwiches and snacks. The owner chooses a 'daily special' sandwich which is displayed on a board outside the shop. Each 'daily special' has two different fillings and is made with one type of bread.

The owner wants a program to randomly choose the 'daily special' sandwich.

The program designer decides to store the possible sandwich fillings in a 1D array of type string.

The array is declared in pseudocode as follows:

```
DECLARE Filling : ARRAY [1:35] OF STRING
```

Each element contains the name of one filling.

An example of the first five elements is as follows:

Index	Element value
1	`"Cheese"`
2	`"Onion"`
3	`"Salmon"`
4	`"Anchovies"`
5	`"Peanut Butter"`

A second 1D array stores the possible bread used:

```
DECLARE Bread : ARRAY [1:10] OF STRING
```

Each element contains the name of one type of bread.

An example of the first three elements is as follows:

Index	Element value
1	`"White"`
2	`"Brown"`
3	`"Pitta"`

Both arrays may contain unused elements. The value of these will be an empty string and they may occur anywhere in each array.

A procedure `Special()` will output a message giving the 'daily special' sandwich made from **two** randomly selected different fillings and **one** randomly selected bread.

Unused array elements must **not** be used when creating the 'daily special' sandwich.

Using the above examples, the output could be:

```
"The daily special is Cheese and Onion on Brown bread."
```

(a) Complete the pseudocode for the procedure `Special()`.

Assume that both arrays are global.

```
PROCEDURE Special()
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
ENDPROCEDURE
```

[7]

BLANK PAGE

8 A program is being developed to implement a game for up to six players.

During the game, each player assembles a team of characters. At the start of the game there are 45 characters available.

Each character has four attributes, as follows:

Attribute	Examples	Comment
Player	0, 1, 3	The player the character is assigned to.
Role	Builder, Teacher, Doctor	The job that the character will perform in the game.
Name	Bill, Lee, Farah, Mo	The name of the character. Several characters may perform the same role, but they will each have a unique name.
Level	14, 23, 76	The skill level of the character. An integer in the range 0 to 100 inclusive.

The programmer has defined a record type to define each character.

The record type definition is shown in pseudocode as follows:

```
TYPE CharacterType
   DECLARE Player : INTEGER
   DECLARE Role : STRING
   DECLARE Name : STRING
   DECLARE Level : INTEGER
ENDTYPE
```

The `Player` field indicates the player to which the character is assigned (1 to 6). The field value is 0 if the character is **not** assigned to any player.

The programmer has defined a global array to store the character data as follows:

```
DECLARE Character : ARRAY[1:45] OF CharacterType
```

At the start of the game all record fields are initialised, and all `Player` field values are set to 0

The programmer has defined a program module as follows:

Module	Description
`Assign()`	• called with two parameters: ○ an integer representing a player ○ a string representing a character role • search the `Character` array for an unassigned character with the required role • If found, assign the character to the given player and output a confirmation message, for example: `"Bill the Builder has been assigned to player 3"` • If no unassigned character with the required role is found, output a suitable message.

(a) Write pseudocode for module `Assign()`.

[7]

(b) A new module will store the contents of the `Character` array in a text file.

The module is defined as follows:

Module	Description
`Save()`	• form a string from each record with fields separated by the character `'^'` • write each string to a separate line of the new file named `SaveFile.txt`

Complete the pseudocode for module `Save()`.

```
PROCEDURE Save()
```

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

```
ENDPROCEDURE
```

[7]

(c) The program is changed and the record type definition is modified as follows:

```
TYPE CharacterType
   DECLARE Player : INTEGER
   DECLARE Role : STRING
   DECLARE Name : STRING
   DECLARE Level : INTEGER
   DECLARE Status : BOOLEAN
ENDTYPE
```

Describe how the additional Boolean field may be stored with the rest of the fields on one line of a text file.

.....

.....

.....

..... [2]

(d) The save operation is to be extended so that multiple files may be saved as the game progresses. This will allow the user to restore the game from any saved position. The filename must reflect the sequence in which the files are saved.

Describe a method that would allow multiple files to be saved **and** give an example of **two** consecutive filenames.

Method .....

.....

Example .....

.....

[2]

Refer to the **insert** for the list of pseudocode functions and operators.

1 A program will calculate the tax payable based on the cost of an item.

Calculations will occur at many places in the program and these involve the use of one of three tax rates.

Tax rate values represent a percentage. For example, a tax rate value of 5.23 represents 5.23%. In this case, the tax payable on an item costing $100 would be $5.23.

Tax rate values are used at several places within the program. One example is given in pseudocode as follows:

```
HighRate ← FALSE
CASE OF ItemCost
   <= 50  : TaxRate ← 3.75      // tax rate of 3.75%
   <= 200 : TaxRate ← 5.23      // tax rate of 5.23%
   > 200  : TaxRate ← 6.25      // tax rate of 6.25%
            HighRate ← TRUE
ENDCASE
TaxPayable ← ItemCost * TaxRate // tax payable
```

(a) The pseudocode contains a logical error.

Identify the error **and** suggest a correction.

Error .....

.....

Correction .....

.....

[2]

(c) Give the **appropriate** data type for the variables in the following table, as used in the pseudocode:

Variable name	Data type
`HighRate`	
`TaxPayable`	

[2]

BLANK PAGE

3 An algorithm will output the **last** three lines from a text file `Result.txt`

The lines need to be output in the same order as they appear in the file.

Assume:

- Three variables `LineX`, `LineY` and `LineZ` will store the three lines. These are of type string and all three variables have been initialised to an empty string.
- The file exists and contains **at least** three lines.

(a) The algorithm to output the lines is expressed in eight steps.

Complete the steps.

1. Open the file .....
2. Loop until .....
3. ..... and store in `ThisLine`
4. Assign `LineY` to `LineX`
5. Assign `LineZ` to `LineY`
6. Assign `ThisLine` to `LineZ`
7. After the loop, ...
8. Output `LineX`, `LineY`, `LineZ`

[4]

(c) The requirement changes, and the algorithm will now output three lines from the file, starting from a **given** line number.

The modified algorithm will be implemented as a function which will:

- be called with an integer parameter representing the given line number
- output three lines, starting at the given line
- return `TRUE` if the 3 lines are output, or `FALSE` if it was not possible to output the 3 lines.

Describe the changes that need to be made to **steps 2 to 8** of the algorithm given in part **(a)**.

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [4]

4 A program includes the following assignment statement:

```
Result ← STR_TO_NUM(x) / STR_TO_NUM(y)
```

When the program evaluates the expression in the statement, it performs a calculation.

Variable `Result` is of type real and variables `x` and `y` are of type string.

Two checks are required before the calculation is performed:

1. The two strings represent valid numeric values.
2. The numeric value of string `y` is not zero.

(a) Identify the type of error that could occur if these checks are **not** carried out **and** state a cause of this error.

Type .....

.....

Cause .....

.....

[2]

(c) The module to perform the checks and calculation will be implemented as a function. The function will need to return both a real **and** a Boolean value. To achieve this a record type is defined in pseudocode as follows:

```
TYPE Result
   DECLARE Done : BOOLEAN
   DECLARE Value : REAL
ENDTYPE
```

The function `Evaluate()` will:

- take two parameters of type string representing the two numeric values
- return a variable of type `Result` with the `Done` field set to `FALSE` if either of the following applies:
  - at least one of the strings does **not** represent a valid numeric value
  - the numeric value of the string representing value `y` is zero
- otherwise return a variable of type `Result` with the `Done` field set to `TRUE` and the `Value` field assigned the result of the formula (based on the numeric value of the two parameters).

Write pseudocode for the function `Evaluate()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

BLANK PAGE

6 A factory produces food items. The items must be used within a certain number of days after their production date. The number of days is known as the shelf life. It is different for each type of item but is always a whole number in the range 1 to 21 (inclusive).

The latest date that an item can be used is called the 'use-by' date.

A program is needed to produce labels which show the 'use-by' date.

Part of the program is a function `GetDate()` which will:

- take two parameters: a production date and a value representing the shelf life
- return the corresponding 'use-by' date.

The program contains a global 1D array `DaysInMonth` of type integer which stores the number of days in each month (index 1 is January):

Index	Value
1	31
2	28
3	31
4	30
11	30
12	31

Note: Leap years are **not** considered

(a) An algorithm uses the array `DaysInMonth` to calculate a 'use-by' date. An alternative design would involve the use of multiple selection statements.

An array-based technique is often used when there is a large number of different values to check and where no pattern exists.

One advantage of using an array-based technique is the speed of execution compared to the use of multiple selection statements.

Give **two other** advantages of using an array for this type of operation rather than a solution based on multiple selection statements.

1 .....

.....

2 .....

.....

[2]

(b) Complete the pseudocode for the function `GetDate()`.

Date functions from the **insert** should be used in your solution.

```
FUNCTION GetDate(ProductionDate : DATE, ShelfLife : INTEGER) RETURNS DATE
```

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

```
ENDFUNCTION
```

[7]

BLANK PAGE

8 An exam paper has a maximum of 75 marks. One of five pass grades (A to E) is assigned, depending on the mark obtained. The lowest mark for a given grade is known as the grade boundary. For example, if the grade boundary for an A grade is 65 marks, then any candidate who achieves a mark of 65 or above will be awarded an A. A grade of U is awarded for marks below the E grade boundary.

The five grade boundaries are stored in a global 1D array `GradeBoundary` of type integer.

For example:

Element	Value	Comment
`GradeBoundary[1]`	65	The minimum mark for an A grade.

A global 2D array `Result` of type integer contains candidate marks for the exam. Each row relates to one candidate. Column 1 contains the candidate mark and column 2 contains the unique candidate ID.

For example, for the fourth and fifth candidates:

Element	Mark
`Result[4, 1]`	56
`Result[5, 1]`	54

Element	ID
`Result[4, 2]`	1074832
`Result[5, 2]`	2573839

There are more rows in the array than candidates who sit the exam. Any unused rows will be at the end of the array.

Candidate papers that are given a mark within two marks of any grade boundary must be checked.

For example, given the values in the example grade boundaries above, any paper that was awarded between 41 and 45 marks (inclusive) would need to be checked.

A program is being written to identify papers that need to be checked.

The programmer has defined the first program module as follows:

Module	Description
`CheckMark()`	• called with a parameter of type integer representing a candidate mark • returns `TRUE` if the mark is within 2 of any of the five grade boundaries, otherwise returns `FALSE`

(a) Write pseudocode for module `CheckMark()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

(b) A second module is defined:

Module	Description
`CheckAll()`	• called with a parameter of type integer representing the number of candidate marks in the `Result` array • uses `CheckMark()` to check each candidate mark • for each paper that needs to be checked, write the corresponding candidate ID on a separate line in a new file named `GRList.txt` • outputs a message with a count of how many papers need to be checked

Write pseudocode for module `CheckAll()`.

`CheckMark()` must be used to check each individual mark.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [8]

(c) The requirement changes. Instead of a new file, the module described in part **(b)** needs to add the corresponding candidate ID for each paper that needs to be checked to an **existing** file.

Explain the change that will need to be made to `CheckAll()`.

.....

..... [1]

GradeBoundary[2]	57	The minimum mark for a B grade.

GradeBoundary[3]	43	The minimum mark for a C grade.

GradeBoundary[4]	35	The minimum mark for a D grade.

GradeBoundary[5]	27	The minimum mark for an E grade.

2 **(a)** A program uses a global 1D array of type string and a text file.

An algorithm that forms part of the program is expressed as follows:
- copy the first line from the file into the first element of the array
- copy the second line from the file into the second element of the array
- continue until all lines in the file have been copied into the array.

Stepwise refinement is applied to the algorithm.

Outline **five** steps for this algorithm that could be used to produce pseudocode.

Assume there are more elements in the array than lines in the file.

Do **not** use pseudocode statements in your answer.

Step 1 .....

.....

Step 2 .....

.....

Step 3 .....

.....

Step 4 .....

.....

Step 5 .....

.....

[5]

BLANK PAGE

3 A record structure is declared to hold data relating to components being produced in a factory:

```
TYPE Component
   DECLARE Item_ID : STRING
   DECLARE Reject : BOOLEAN
   DECLARE Weight : REAL
ENDTYPE
```

The factory normally produces a batch (or set) of 1000 components at a time. A global array is declared to store 1000 records for a batch:

```
DECLARE Batch : ARRAY [1:1000] OF Component
```

Two global variables contain the minimum and maximum acceptable weight for each component. The values represent an inclusive range and are declared as:

```
DECLARE Min, Max : REAL
```

(a) (i) A program uses a variable `ThisIndex` as the array index to access a record.

Write a pseudocode clause to check whether or **not** the weight of an individual component is within the acceptable range.

.....

.....

..... [3]

(ii) When batches of less than 1000 components are processed, it is necessary to indicate that certain elements in the array are unused.

Suggest how an unused array element could be indicated.

.....

..... [1]

4 A procedure `TwoParts()` will input a sequence of real values, one at a time.

The procedure will:
- process the sequence in two parts
- form a first total by adding the values until the first zero
- form a second total by adding the values after the first zero until the second zero
- output the average of the two totals, together with a suitable message.

Values input in the first part are totalled using global variable `TotalA` and those input in the second part are totalled using global variable `TotalB`.

(a) Write pseudocode for the procedure `TwoParts()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

(b) The value zero denotes the split between the two parts of the sequence.

The requirement changes and now there may be up to 20 parts.

(i) Identify a suitable data structure that could be used to store the different total values.

..... [2]

(ii) Describe **three** benefits of using the data structure given in part **(b)(i)**.

1 .....

.....

2 .....

.....

3 .....

.....

[3]

8 A teacher is designing a program to process pseudocode projects written by her students.

The program analyses a student project and extracts information about each module that is defined (each procedure or function). This information is stored in a global 2D array `ModInfo` of type string.

A module header is the first line of a module definition and starts with either of the keywords `PROCEDURE` or `FUNCTION`.

An example of part of the array is given below. Row 10 of the array shows that a procedure header occurs on line 27 and row 11 shows that a function header occurs on line 35. `"P"` represents a procedure and `"F"` represents a function:

	x = 1	**x = 2**	**x = 3**
`ModInfo[10, x]`	`"27"`	`"P"`	`"MyProc(Z : CHAR)"`
`ModInfo[11, x]`	`"35"`	`"F"`	`"MyFun(Y : CHAR) RETURNS BOOLEAN"`

The string stored in column 3 is called the module description. This is the module header **without** the keyword.

A valid module header will:
- be at least 13 characters long
- start with the keyword `PROCEDURE` or `FUNCTION`. The keyword may appear in either upper or lower case (or a mix of both) and **must** be followed by a space character.

The teacher has defined the first program module as follows:

Module	Description

`Header()`	• called with a parameter of type string representing a line of pseudocode
	• if the line is a valid procedure header, returns a string:
	`"P<Module description>"`
	• if the line is a valid function header, returns a string:
	`"F<Module description>"`
	• otherwise, returns an empty string
	
	For example, given the string:
	
	`"FUNCTION Zap(X : INTEGER) RETURNS CHAR"`
	
	`Header()` returns the string:
	
	`"FZap(X : INTEGER) RETURNS CHAR"`

(a) Write pseudocode for module `Header()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [7]

(b) A new module is required:

Module	Description
`FindModules()`	• called with a parameter of type string representing a student project file name • uses module `Header()` to check each line of the project • assigns values to the `ModInfo` array for each module declaration in the student project

As a reminder, the previous example of part of the array is repeated below:

	x = 1	**x = 2**	**x = 3**
`ModInfo[10, x]`	`"27"`	`"P"`	`"MyProc(Z : CHAR)"`
`ModInfo[11, x]`	`"35"`	`"F"`	`"MyFun(Y : CHAR) RETURNS BOOLEAN"`

Write pseudocode for module `FindModules()`.

Assume that the array contains enough rows for the number of modules in each project.

.....

.....

.....

.....

.....

.....

.....

..... [8]

Refer to the **insert** for the list of pseudocode functions and operators.

1 (a) The following table contains pseudocode examples.

Each example may contain statements that relate to one or more of the following:
- selection
- iteration (repetition)
- input/output.

Complete the table by placing **one or more** ticks (✓) in each row.

Pseudocode example	Selection	Iteration	Input/Output

`FOR Index ← 1 TO 10`
`  Data[Index] ← 0`
`NEXT Index`			

`WRITEFILE ThisFile, "****"`			
`UNTIL Level > 25`			

`IF Mark > 74 THEN`
`  READFILE OldFile, Data`
`ENDIF`			

[4]

(b) Program variables have data types as follows:

Variable	Data type
`MyChar`	`CHAR`
`MyString`	`STRING`
`MyInt`	`INTEGER`

Complete the table by filling in each gap with a function (from the **insert**) so that each expression is valid.

Expression
`MyInt ← ..... (3.1415926)`
`MyChar ← ..... ("Elwood", 3, 1)`
`MyString ← ..... ( ..... (27.509))`
`MyInt ← ..... ( ..... ("ABC123", 3))`

[4]

3 A factory needs a program to help manage its production of items.

Data will be stored about each item.

The data for each item will be held in a record structure of type `Component`.

The programmer has started to define the fields that will be needed as shown in the table.

Field	Example value	Comment
`Item_Num`	`123478`	a numeric value used as an array index
`Reject`	`FALSE`	`TRUE` if this item has been rejected
`Stage`	`'B'`	a letter to indicate the stage of production
`Limit_1`	`13.5`	any value in the range 0 to 100 inclusive
`Limit_2`	`26.4`	any value in the range 0 to 100 inclusive

(a) (i) Write pseudocode to declare the record structure for type `Component`.

.....

.....

.....

.....

.....

.....

..... [4]

(ii) A 1D array `Item` of 2000 elements will store the data for all items.

Write pseudocode to declare the `Item` array.

.....

..... [2]

(b) State **three** benefits of using an array of records to store the data for all items.

1 .....

.....

2 .....

.....

3 .....

.....

[3]

6 A music player stores music in a digital form and has a display which shows the track being played.

(a) Up to 16 characters can be displayed. Track titles longer than 16 characters will need to be trimmed as follows:

- Words must be removed from the end of the track title until the resulting title is less than 14 characters.
- When a word is removed, the space in front of that word is also removed.
- Three dots are added to the end of the last word displayed when one or more words have been removed.

The table below shows some examples:

Original title	Display string															
	1	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**	**16**
Bat out of Hull	**B**	**a**	**t**		**o**	**u**	**t**		**o**	**f**		**H**	**u**	**l**	**l**	
Bohemian Symphony	**B**	**o**	**h**	**e**	**m**	**i**	**a**	**n**	**.**	**.**	**.**					
Paperbook Writer	**P**	**a**	**p**	**e**	**r**	**b**	**o**	**o**	**k**		**W**	**r**	**i**	**t**	**e**	**r**
Chris Sings the Blues	**C**	**h**	**r**	**i**	**s**		**S**	**i**	**n**	**g**	**s**	**.**	**.**	**.**		
Green Home Alabama	**G**	**r**	**e**	**e**	**n**		**H**	**o**	**m**	**e**	**.**	**.**	**.**			

A function `Trim()` will:

- take a string representing the original title
- return the string to be displayed.

Assume:

- Words in the original title are separated by a single space character.
- There are no spaces before the first word or after the last word of the original title.
- The first word of the original title is less than 14 characters.

Write pseudocode for the function `Trim()`.

.....

.....

.....

(b) Music is stored as a sequence of digital samples.

Each digital sample is a denary value in the range 0 to 99999999 (8 digits).

The samples are to be stored in a text file. Each sample is converted to a numeric string and 32 samples are concatenated (joined) to form a single line of the text file.

Each numeric string is 8 characters in length; leading '0' characters are added as required.

Example:

Sample	Denary value	String
1	456	"00000456"
2	48	"00000048"
3	37652	"00037652"
32	673	"00000673"

The example samples will be stored in the text file as a single line:

"000004560000004800037652...00000673"

(i) Identify one drawback of adding leading '0' characters to each numeric string.

.....

..... [1]

(ii) Suggest an alternative method of storing the samples which does **not** involve adding leading '0' characters but which would still allow each individual sample to be extracted.

.....

.....

..... [1]

(iii) State **one** drawback of the alternative method given in part **(b)(ii)**.

.....

..... [1]

8 A teacher is designing a program to process pseudocode projects written by her students.

Each student project is stored in a text file.

The process is split into a number of stages. Each stage performs a different task and creates a new file.

For example:

File name	Comment
`MichaelAday_src.txt`	Student project file produced by student Michael Aday
`MichaelAday_S1.txt`	File produced by stage 1
`MichaelAday_S2.txt`	File produced by stage 2

(a) Suggest a reason why the teacher's program has been split into a number of stages **and** give the benefit of producing a different file from each stage.

Reason .....

.....

Benefit .....

.....

[2]

(b) The teacher has defined the first program module as follows:

Module	Description
`DeleteSpaces()`	• called with a parameter of type string representing a line of pseudocode from a student's project file • returns the line after removing any leading space characters The following example shows a string before and after the leading spaces have been removed: Before: `"   IF X2 > 13 THEN"` After: `"IF X2 > 13 THEN"`

Complete the pseudocode for module `DeleteSpaces()`.

```
FUNCTION DeleteSpaces(Line : STRING) RETURNS STRING
```

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

```
ENDFUNCTION
```

[6]

(c) Two modules are defined:

Module	Description
`DeleteComment()` (already written)	• called with a parameter of type string representing a line of pseudocode from a student's project file • returns the line after removing any comment
`Stage_2()`	• called with two parameters: ○ a string representing an input file name ○ a string representing an output file name • copies each line from the input file to the existing output file having first removed all leading spaces and comments from that line • does not write blank lines to the output file • outputs a final message giving the number of blank lines removed

Write pseudocode for module `Stage_2()`.

Modules `DeleteComment()` and `DeleteSpaces()` must be used in your solution.

..... [8]

Refer to the **insert** for the list of pseudocode functions and operators.

1 An algorithm is developed in pseudocode before being coded in a programming language.

(a) The following table shows four valid pseudocode assignment statements.

Complete the table by giving an appropriate data type to declare each of the variables `A`, `B`, `C` and `D`.

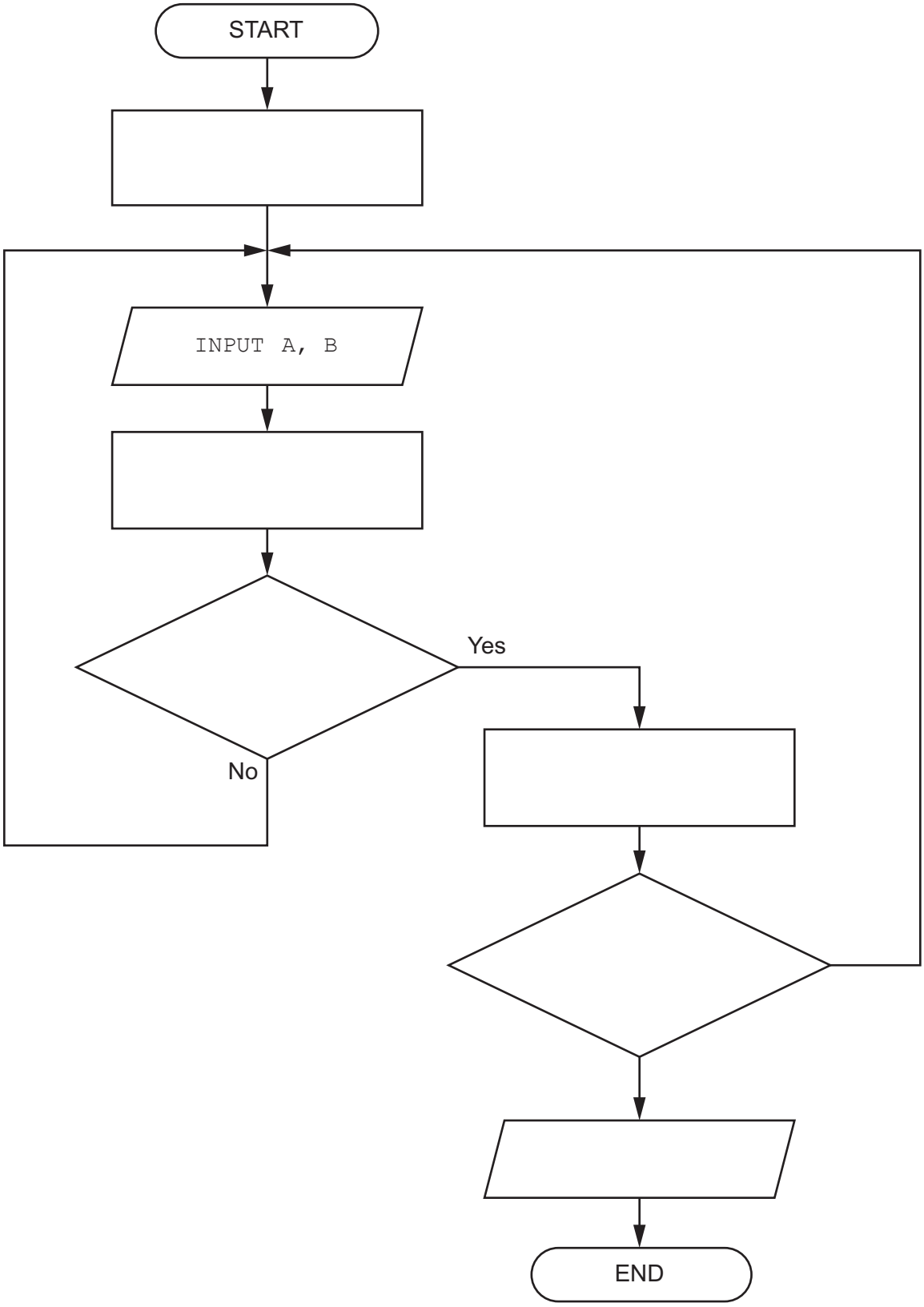
Assignment statement	Data type
`A ← LEFT(MyName, 1)`	
`B ← Total * 2`	
`C ← INT(ItemCost) / 3`	
`D ← "Odd OR Even"`	

[4]

2 An algorithm has three steps. It will:

1. repeatedly input a pair of numeric values `A` and `B`
2. count the number of pairs that are input until `A` has been greater than `B` 10 times
3. output the number of pairs that were input.

(a) Complete the program flowchart.



[5]

(b) Step 1 of the algorithm is changed.

A variable `ThisSequence` is used to enter a sequence of 10 pairs of numeric values, using a single input statement.

Following the input of `ThisSequence` the revised algorithm will extract the pairs of numbers.

Describe the variable `ThisSequence` and how the numbers are extracted.

.....

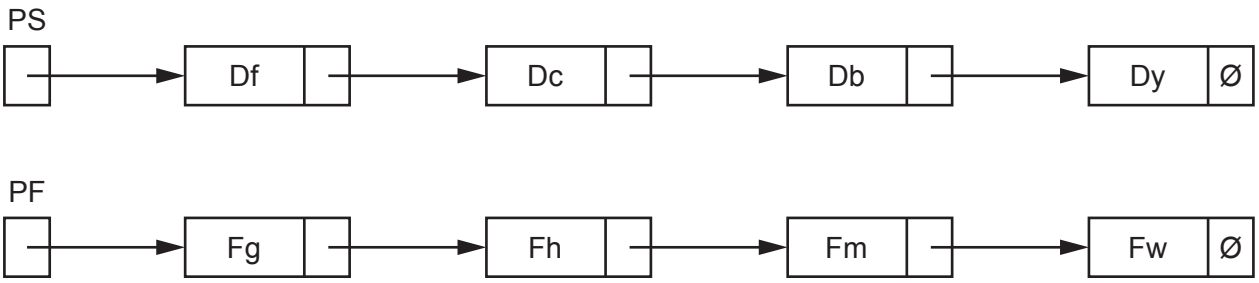
.....

.....

..... [2]

3 The diagram shows an Abstract Data Type (ADT) representation of a linked list after data items have been added.

- PS is the start pointer.
- PF is the free list pointer.
- Labels Df, Dc, Db and Dy represent the data items of nodes in the list.
- Labels Fg, Fh, Fm and Fw represent the data items of nodes in the free list.
- The symbol Ø represents a null pointer.



(a) Describe the linked list immediately after initialisation, before **any** data items are added.

.....

.....

.....

.....

.....

..... [3]

(b) A program will be written to include a linked list to store alphanumeric user IDs.

The design uses two variables and two 1D arrays to implement the linked list.
Each array element contains data of a single data type and **not** a record.

The statements below describe the design.

Complete the statements.

The two variables will be of type ..... .

The two variables will be used as ..... to the arrays.

The values stored in the two variables will indicate .....

..... .

The first 1D array will be of type ..... .

The first 1D array will be used to ..... .

The second 1D array will be of type ..... .

The second 1D array will be used to ..... .

[5]

4 A global 1D array `Data` contains 100 elements of type integer.

A function `Check()` will:

- total the element values in odd index locations (1, 3, 5 ... 97, 99)
- total the element values in even index locations (2, 4, 6 ... 98, 100)
- return one of three strings 'Odd', 'Even' or 'Same' to indicate which total is the greater, or whether the totals are the same.

Write pseudocode for the function `Check()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

5 A global 1D array of strings contains three elements which are assigned values as shown:

```
Data[1] ← "aaaaaa"
Data[2] ← "bbbbbb"
Data[3] ← "cccccc"
```

Procedure `Process()` manipulates the values in the array.

The procedure is written in pseudocode as follows:

```
PROCEDURE Process(Format : STRING)
   DECLARE Count, Index, L : INTEGER
   DECLARE Result : STRING
   DECLARE C : CHAR

   Result ← "****"

   FOR Count ← 1 TO LENGTH(Format) STEP 2
      C ← MID(Format, Count, 1)
      L ← STR_TO_NUM(MID(Format, Count + 1, 1))

      Index ← (Count + 1) DIV 2

      CASE OF C
         'X' : Result ← TO_UPPER(Data[Index])
         'Y' : Result ← TO_LOWER(Data[Index])
         'Z' : Result ← "**" & Data[Index]
      ENDCASE

      Data[Index] ← LEFT(Result, L)
   NEXT Count

ENDPROCEDURE
```

(a) Complete the trace table by dry running the procedure when it is called as follows:

```
CALL Process("X3Y2W4")
```

Count	C	L	Index	Result	Data[1]	Data[2]	Data[3]

[6]

8 A teacher is designing a program to process pseudocode projects written by her students.

Each student project is stored in a text file.

The process is split into a number of stages. Each stage performs a different task and creates a new file named as shown:

File name	Comment
`MichaelAday_src.txt`	student project file produced by student Michael Aday
`MichaelAday_S1.txt`	file produced by stage 1
`MichaelAday_S2.txt`	file produced by stage 2

The teacher has defined the first program module as follows:

Module	Description
`DeleteComment()`	• called with a parameter of type string representing a line of pseudocode from a student's project file • returns the line after removing any comments Note on comments: A comment starts with two forward slash characters and includes all the remaining characters on the line. The following example shows a string before and after the comment has been removed: Before: `IF X2 > 13 THEN   //check if limit exceeded` After: `IF X2 > 13 THEN`

(a) Complete the pseudocode for module `DeleteComment()`.

```
FUNCTION DeleteComment(Line : STRING) RETURNS STRING
```

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

```
ENDFUNCTION
```

[8]

(b) A second module is defined:

Module	Description
`Stage_1()`	• called with a parameter of type string representing a student name • creates a new stage 1 file • copies each line from the student's project file to the stage 1 file after removing any comment from each line • does **not** write blank lines to the stage 1 file • returns the number of lines written to the stage 1 file

Write pseudocode for module `Stage_1()`.

Module `DeleteComment()` must be used in your solution.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [7]

Refer to the **insert** for the list of pseudocode functions and operators.

1 A program is being developed in pseudocode before being converted into a programming language.

(a) The following table shows four valid pseudocode assignment statements.

Complete the table by giving the data type that should be used to declare the variable underlined in each assignment statement.

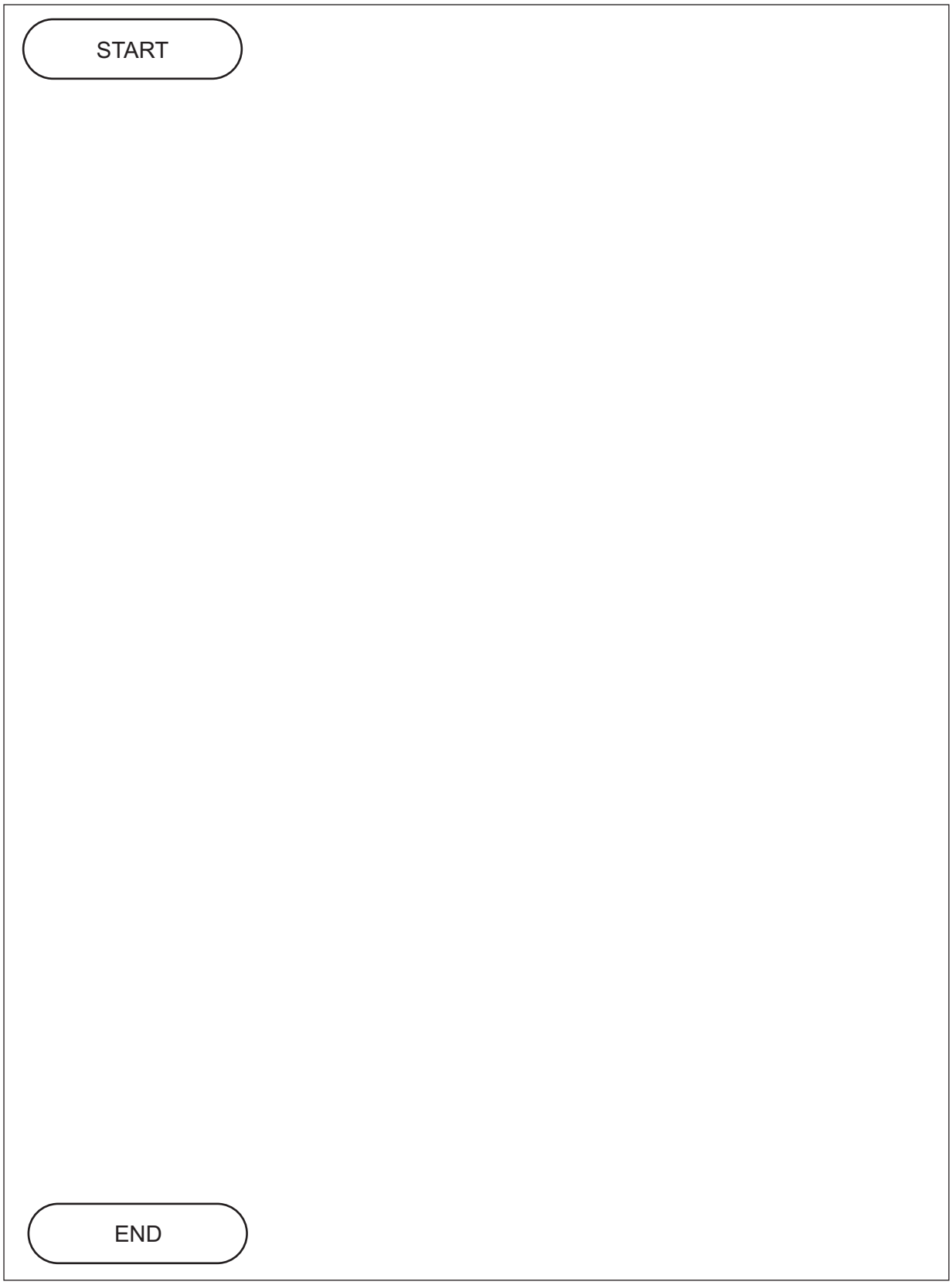
Assignment statement	Data type
MyVar1 ← Total1 / Total2	
MyVar2 ← 27/10/2023	
MyVar3 ← "Sum1 / Sum2"	
MyVar4 ← Result1 AND Result2	

[4]

2 `Data` is a 1D array of integers, containing 30 elements. All element values are unique.

(a) An algorithm will output the index of the element with the smallest value.

Draw a program flowchart to represent the algorithm.



[5]

(b) The 30 data values could have been stored in separate variables rather than in an array.

Explain the benefits of using an array when designing a solution to part **(a)**.

.....

..... [2]

(c) The requirement changes. Array `Data` needs to hold 120 elements and each value may include a decimal place.

Write a pseudocode statement to declare the modified array.

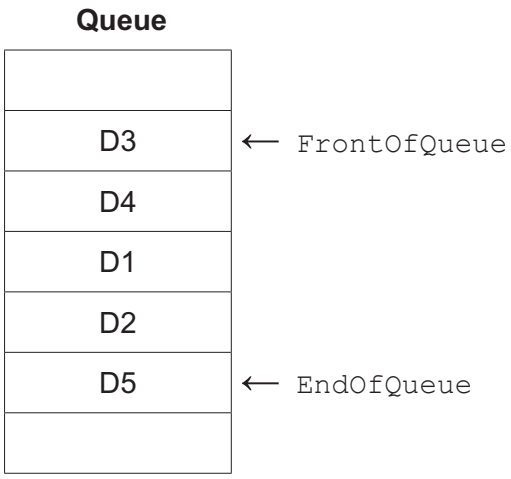
.....

..... [2]

3 The diagram represents a queue Abstract Data Type (ADT).

The organisation of this queue may be summarised as follows:

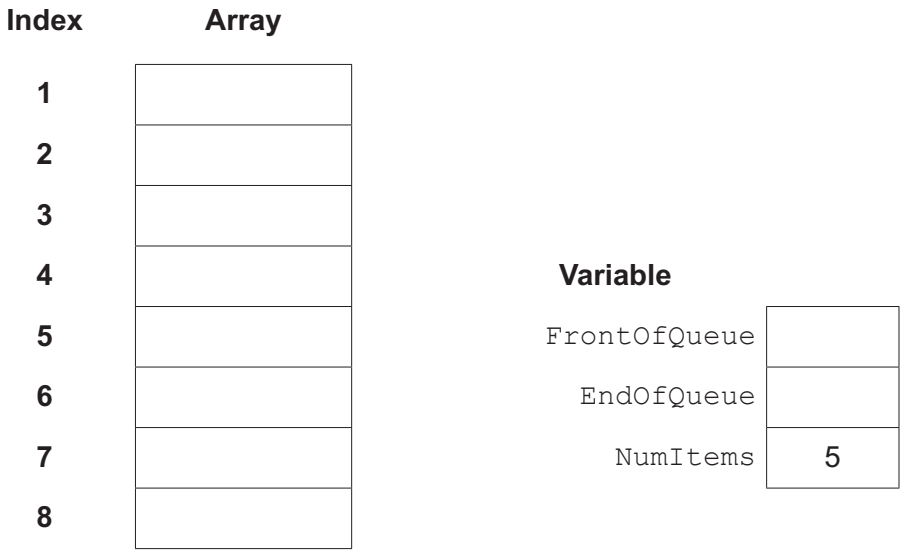
- The `FrontOfQueue` pointer points to the next data item to be removed.
- The `EndOfQueue` pointer points to the last data item added.



The queue is implemented using three variables and a 1D array of eight elements as shown. The variable `NumItems` stores the number of items in the queue.

The pointer variables store indices (index numbers) of the array.

(a) Complete the diagram to represent the state of the queue as shown above.



[3]

(b) A module `AddTo()` will add a value to the queue by manipulating the array and variables in part **(a)**.

The queue implementation is circular. When pointers reach the end of the queue, they will 'wrap around' to the beginning.

Before a value can be added to the queue, it is necessary to check the queue is not full.

The algorithm to add a value to the queue is expressed in six steps.

Complete the steps.

1. If `NumItems` ..... then jump to step 6.
2. Increment ..... .
3. If ..... then set `EndOfQueue` to ..... .
4. Increment ..... .
5. Set the ..... at the index stored in ..... to the ..... being added.
6. Stop.

[6]

5 A global 1D array of integers contains four elements, which are assigned values as shown:

```
Mix[1] ← 4
Mix[2] ← 2
Mix[3] ← 3
Mix[4] ← 5
```

A procedure `Process()` manipulates the values in the array.

The procedure is written in pseudocode as follows:

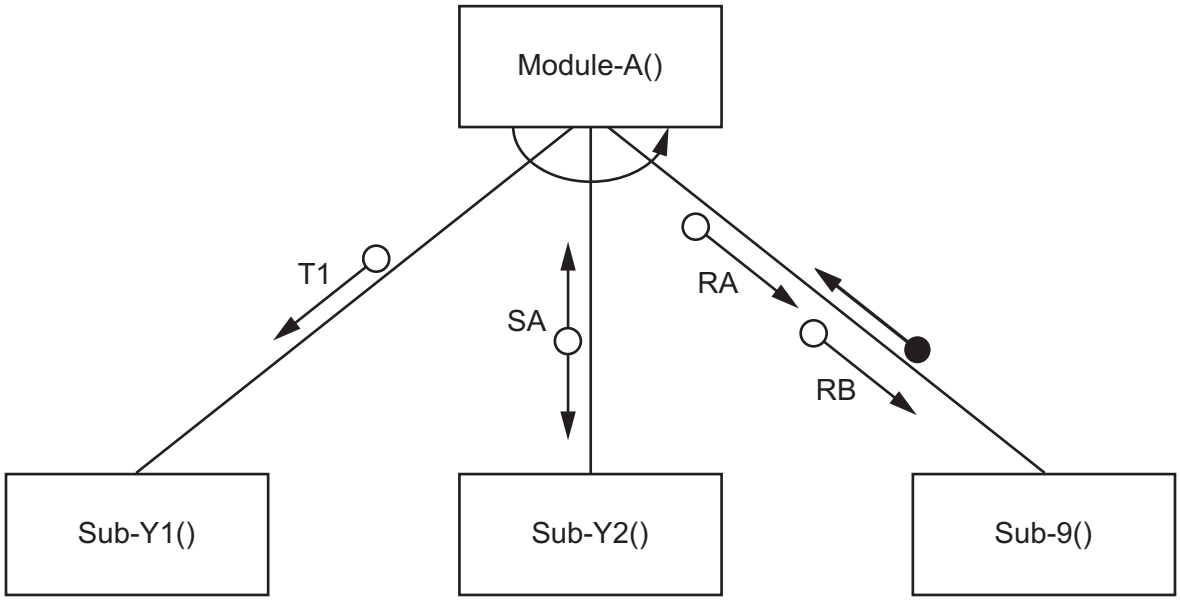
```
PROCEDURE Process(Start : INTEGER)
   DECLARE Value, Index, Total : INTEGER

   Index ← Start
   Total ← 0

   WHILE Total < 20
      Value ← Mix[Index]
      Total ← Total + Value

      IF Index < 4 THEN
         Mix[Index] ← Mix[Index] + Mix[Index+1]
      ELSE
         Mix[Index] ← Mix[Index] + Mix[1]
      ENDIF
```

7 A structure chart shows the modular structure of a program:



(a) Explain the meaning of the curved arrow symbol which begins and ends at Module-A().

.....

.....

.....

..... [2]

(b) The structure chart shows that Sub-9() is a function.

A Boolean value is returned by Sub-9() for processing by Module-A().

The original parameter RA is of type integer and RB is of type string.

A record type `MyType` will be defined with three fields to store the values passed between the two modules.

(i) Write pseudocode to define `MyType`.

.....

.....

.....

.....

.....

..... [3]

8 A class of students are developing a program to send data between computers. Many computers are connected together to form a wired network. Serial ports are used to connect one computer to another.

Each computer:
- is assigned a unique three-digit ID
- has three ports, each identified by an integer value
- is connected to between one and three other computers.

Messages are sent between computers as a string of characters organised into fields as shown:

```
<STX><DestinationID><SourceID><Data><ETX>
```

Field name	Description
`STX`	a single character marking the start of the message (ASCII value 02)
`DestinationID`	three numeric characters identifying the destination computer
`SourceID`	three numeric characters identifying the source computer
`Data`	a variable length string containing the data being sent (Minimum length is 1 character)
`ETX`	a single character marking the end of the message (ASCII value 03)

For example, the following message contains the data "Hello Jack" being sent from computer "202" to computer "454":

```
<STX>"454202Hello Jack"<ETX>
```

Each computer will run a copy of the same program. Each program will contain a global variable `MyID` of type string which contains the unique ID of the computer in which the program is running.

The first two program modules are defined as follows:

Module	Description
`GetData()` (already written)	• returns the data field from a message that has been received • If no message is available, the module waits until one has been received.
`ReceiveFile()`	• takes a file name as a parameter of type string • creates a text file with the given file name (no checking required) • writes the data field returned by `GetData()` to the file • repeats until the data field is "****", which is not written to the file • outputs a final message giving the total number of characters written to the file, for example: `132456 characters were written to newfile.txt`

(a) Write pseudocode for module `ReceiveFile()`.

Module `GetData()` has already been written and must be used.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [7]

(b) The use of the string "****" as explained in the module description for `ReceiveFile()` may cause a problem.

Explain the problem and suggest a solution.

Problem .....

.....

.....

.....

Solution .....

.....

.....

.....

[3]

```
        Index ← (Value MOD 4) + 1
    ENDWHILE
    Mix[1] ← Total * Index
ENDPROCEDURE
```

Complete the trace table on the opposite page by dry running the procedure when it is called as follows:

```
CALL Process(2)
```

Index	Value	Total	Mix[1]	Mix[2]	Mix[3]	Mix[4]

Refer to the **insert** for the list of pseudocode functions and operators.

1 A shop sells car accessories. A customer order is created if an item cannot be supplied from current stock. A program is being developed to create and manage the customer orders.

(a) The following identifier table shows some of the data that will be stored for each order.

Complete the identifier table by adding meaningful variable names and appropriate data types.

Example value	Explanation	Variable name	Data type
`"Mr Khan"`	The name of the customer		
`3`	The number of items in the order		
`TRUE`	To indicate whether this is a new customer		
`15.75`	The deposit paid by the customer		

[4]

(c) The data that needs to be stored for each customer order in part **(a)** is not all of the same type.

Describe an effective way of storing this data for many customer orders while the program is running.

.....

.....

.....

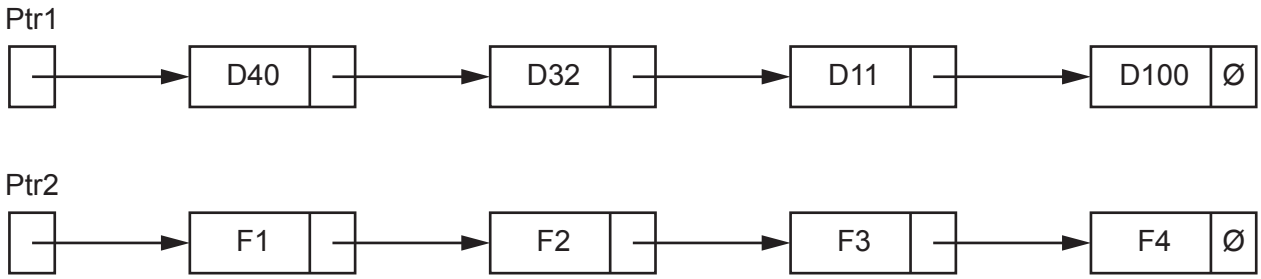
.....

.....

..... [3]

3 The diagram represents a linked list Abstract Data Type (ADT).

- Ptr1 is the start pointer. Ptr2 is the free list pointer.
- Labels D40, D32, D11 and D100 represent the data items of nodes in the list.
- Labels F1, F2, F3 and F4 represent the data items of nodes in the free list.
- The symbol Ø represents a null pointer.



(a) The linked list is implemented using two variables and two 1D arrays as shown.

The pointer variables and the elements of the Pointer array store the indices (index numbers) of elements in the Data array.

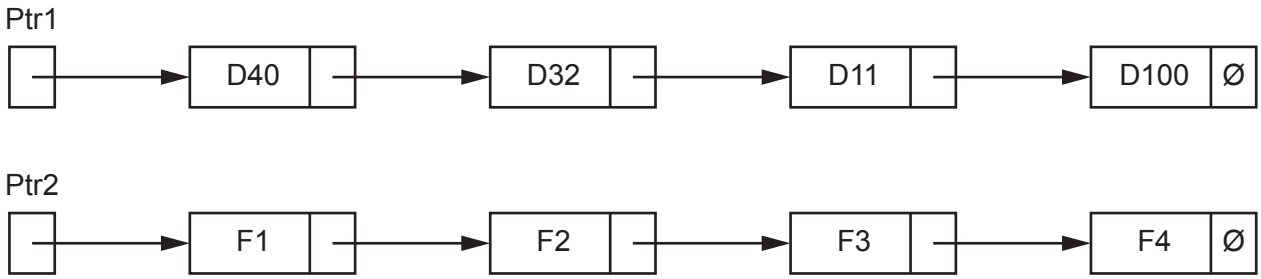
Complete the diagram to show how the linked list as shown above may be represented using the variables and arrays.

Variable	Value
`Start_Pointer`	
`Free_List_Pointer`	5

Index	Data array	Pointer array
1	D32	2
2		3
3		
4	D40	
5		
6	F2	7
7		
8		

[5]

(b) The original linked list is to be modified. A new node D6 is inserted between nodes D32 and D11.



The algorithm required is expressed in four steps as shown.

Complete the steps.

1. Assign the data item ..... to ..... .
2. Set the ..... of this node to point to ..... .
3. Set Ptr2 to point to ..... .
4. Set pointer of ..... to point to ..... .

[4]

5 A global 1D array of integers contains four elements, which are assigned values as shown:

```
Mix[1] ← 1
Mix[2] ← 3
Mix[3] ← 4
Mix[4] ← 2
```

A procedure `Process()` manipulates the values in the array.

The procedure is written in pseudocode:

```
PROCEDURE Process(Start : INTEGER)
   DECLARE Value, Index, Count : INTEGER

   Index ← Start
   Count ← 0

   REPEAT
      Value ← Mix[Index]
      Mix[Index] ← Mix[Index] - 1
      Index ← Value
      Count ← Count + 1
   UNTIL Count = 5

   Mix[4] ← Count * Index

ENDPROCEDURE
```

Complete the trace table on the opposite page by dry running the procedure when it is called as follows:

```
CALL Process(3)
```

Index	Value	Count	Mix[1]	Mix[2]	Mix[3]	Mix[4]

[6]

6 **(a)** A procedure `CreateFiles()` will take two parameters:

- a string representing a file name
- an integer representing the number of files to be created.

The procedure will create the number of text files specified.

Each file is given a different name. Each file name is formed by concatenating the file name with a suffix based on the file number. The suffix is always three characters.

For example, the call `CreateFiles("TestData", 3)` would result in the creation of the three files, TestData.001, TestData.002 and TestData.003.

Each file will contain a single line. For example, file TestData.002 would contain the string:

```
This is File TestData.002
```

Write pseudocode for `CreateFiles()`.

Assume both parameters are valid and that the integer value is between 1 and 999, inclusive.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

(b) A module `CheckFiles()` will count the number of files produced by `CreateFiles()` in part **(a)**.

`CheckFiles()` will take a string representing a file name and return the number of files found.

(i) Identify the type of module that should be used for `CheckFiles()`.

..... [1]

(iii) State the file mode that should be used in `CheckFiles()`.

..... [1]

8 A class of students are developing a program to send data between computers. Many computers are connected together to form a wired network. Serial ports are used to connect one computer to another.

Each computer:

- is assigned a unique three-digit ID
- has three ports, each identified by an integer value
- is connected to between one and three other computers.

Data is sent as individual message strings.
Each string contains the destination ID (the ID of the computer that is to receive the message) followed by the data:

```
<DestinationID><Data>
```

Messages may pass through several computers on the way to their destination.
When a message arrives at a computer, that is **not** the destination, the program needs to forward it on to another computer using one of its serial ports.

The port to use is obtained from information that is stored in an array `RouteTable`.

`RouteTable` is a global 2D array of integers. It is declared in pseudocode as follows:

```
DECLARE RouteTable : ARRAY[1:6,1:3] OF INTEGER
```

The values in the first two columns of `RouteTable` define a range of ID values.
Column 3 gives the corresponding port number to use when forwarding the message to a computer with an ID within this range.

For example, the contents of `RouteTable` could be:

	Column 1	**Column 2**	**Column 3**
Row 1	100	199	1
Row 2	200	259	2
Row 3	−1	<undefined>	<undefined>
Row 4	260	399	2
Row 5	400	599	3
Row 6	600	999	1

In this example, a message that arrives with a `DestinationID` of "283" will be forwarded using port 2.

Row 3 in the example shows an unused row. These may occur anywhere. Unused rows have the column 1 element set to −1. The value of unused elements in the other two columns is undefined.

The programmer has defined the first program module as follows:

Module	Description
`GetPort()`	• takes a `DestinationID` as a parameter of type string • searches for the range corresponding to the `DestinationID` in the array • returns the port number, or returns −1 if no corresponding range is found

(a) Write pseudocode for module `GetPort()`.

Assume `DestinationID` contains a valid three-digit string.

..... [7]

(b) Copies of the same program will run on each computer. The program contains a global variable `MyID` of type string, which contains the unique ID of the computer in which the program is running.

When messages are received, they are placed on one of two stacks. Stack 1 is used for messages that have reached their destination and stack 2 is used for messages that will be forwarded on to another computer.

Additional modules are defined:

Module	Description
`StackMsg()` (already written)	• takes two parameters: ○ a string representing a message ○ an integer representing the stack number • adds the message to the required stack • returns `TRUE` if the message is added to the required stack, otherwise returns `FALSE`
`ProcessMsg()`	• takes a message as a parameter of type string • ignores any message with a zero-length data field • extract the `DestinationID` from the message • checks whether the `DestinationID` is this computer or whether the message is to be forwarded • uses `StackMsg()` to add the message to the appropriate stack • outputs an error if the message could not be added to the stack

Write pseudocode for module `ProcessMsg()`.

Module `StackMsg()` must be used.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [7]

(c) The program contains a module `GetFile()` which receives text files sent from another computer.

Lines from the file are sent one at a time. Each message contains one line and `ProcessMsg()` from part **(b)** adds each message as it is received onto stack 1.

Module `GetFile()` removes messages from stack 1 and writes the data to a text file.

There is a problem. Under certain circumstances, the received file does not appear as expected.

Assume that while a file is being received `ProcessMsg()` receives only messages containing lines from the file.

(i) Describe the circumstances and explain the problem.

Circumstances .....

.....

Explanation .....

.....

.....

.....

[3]

(ii) Suggest a more appropriate Abstract Data Type that could be used to store the messages that would not have the same problem.

..... [1]

Refer to the **insert** for the list of pseudocode functions and operators.

1 The following pseudocode represents part of the algorithm for a program:

```
CASE OF ThisValue
   < 30       : Level ← "Low"           // less than 30
                Check ← 1
   < 20       : Level ← "Very Low"      // less than 20
                Check ← ThisValue / 2
   30 TO 40   : Level ← "Medium"        // between 30 and 40
                Check ← ThisValue / 3
                Data[ThisValue] ← Data[ThisValue] + 1
   > 40       : Level ← "High"
ENDCASE
```

(a) Complete the table by writing the answer for each row:

	Answer
The value assigned to `Level` when `ThisValue` is 40	
The value assigned to `Check` when `ThisValue` is 36	
The value assigned to `Level` when `ThisValue` is 18	
The number of elements in array `Data` that may be incremented	

[4]

(d) Give the appropriate data types for the variables `ThisValue`, `Check` and `Level`.

`ThisValue` .....

`Check` .....

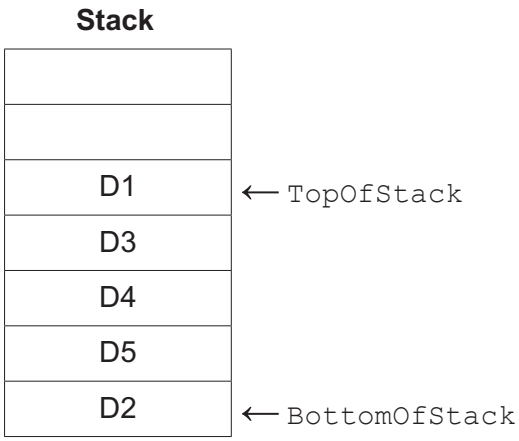
`Level` .....

[3]

3 The diagram represents an Abstract Data Type (ADT).

The operation of this stack may be summarised as follows:

- The `TopOfStack` pointer points to the last item added to the stack.
- The `BottomOfStack` pointer points to the first item on the stack.

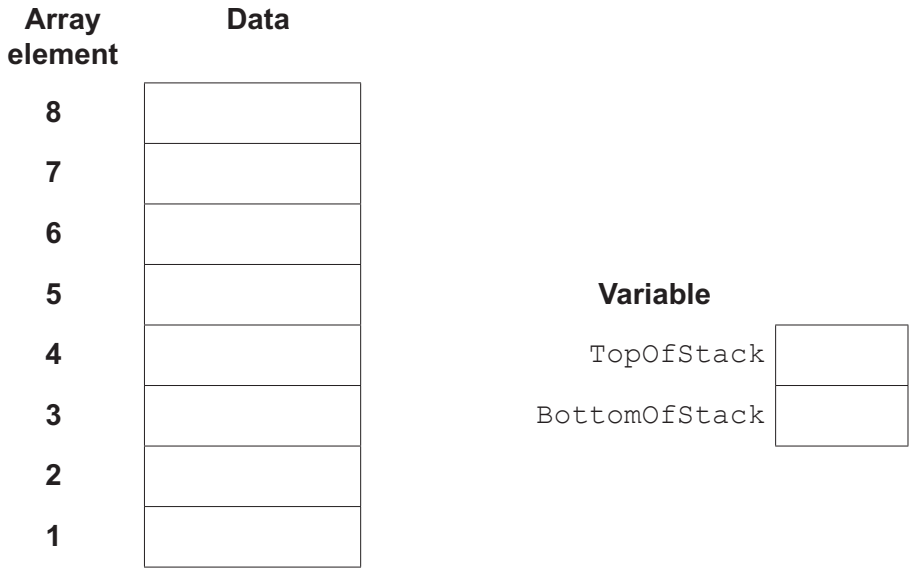


(a) The stack is implemented using two variables and a 1D array of 8 elements as shown.

The variables are used to reference individual elements of the array, in such a way that:

- the array is filled from the lowest indexed element towards the highest
- all the elements of the array are available for the stack.

Complete the diagram to represent the state of the stack as shown above.



[3]

(b) A function `Push()` will add a value onto the stack by manipulating the array and variables in **part (a)**.

Before adding a value onto the stack, the algorithm will check that space is available.

If the value is added to the stack, the function will return `TRUE`, otherwise it will return `FALSE`.

The algorithm is expressed in five steps.

Complete the steps.

1. If ..... then return `FALSE`
2. Otherwise ..... `TopOfStack`
3. Use `TopOfStack` as an ..... to the array.
4. Set the element at this ..... to the ..... being added.
5. Return ..... .

[5]

4 A global array is declared in pseudocode as follows:

```
DECLARE Data : ARRAY[1:150] OF STRING
```

A function `TooMany()` will:

1. take two parameters:
   - a string (the search string)
   - an integer (the maximum value)
2. count the number of strings in the array that exactly match the search string
3. return `TRUE` if the count is greater than the maximum value, otherwise will return `FALSE`

(a) Write pseudocode for the function `TooMany()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

(b) The global array is changed to a 2D array, organised as 150 rows by 2 columns. It is declared in pseudocode as follows:

```
DECLARE Data : ARRAY[1:150, 1:2] OF STRING
```

The algorithm for the function in **part (a)** is changed. Strings will only be counted if **both** of the following conditions are true:

- The current row is an even number.
- The search string exactly matches the value in **either** column.

Write pseudocode to check these conditions.

Assume that the row index is contained in variable `Row` and the search string in variable `Search`.

.....

.....

.....

.....

.....

..... [3]

8 A class of students are developing a program to send data between computers. Many computers are connected together to form a wired network. Serial ports are used to connect one computer to another.

Each computer:

- is assigned a unique three-digit ID
- has three ports, each identified by an integer value
- is connected to between one and three other computers.

Messages are sent between computers as a string of characters organised into fields as shown:

```
<STX><DestinationID><SourceID><Data><ETX>
```

Field number	Field name	Description
n/a	`STX`	a single character marking the start of the message (ASCII value 02)
1	`DestinationID`	three numeric characters that identify the destination computer
2	`SourceID`	three numeric characters that identify the source computer
3	`Data`	a variable length string containing the data being sent (Minimum length is 1 character)
n/a	`ETX`	a single character marking the end of the message (ASCII value 03)

For example, the following message contains the data "Hello Kevin" being sent from computer "101" to computer "232":

```
<STX>"232101Hello Kevin"<ETX>
```

Each computer will run a copy of the same program. Each program will contain a global variable, `MyID` of type string, that contains the unique ID of the computer in which the program is running.

The programmer has defined the first two program modules as follows:

Module	Description
`Transmit()` (already written)	• takes two parameters: ○ a string containing a message ○ an integer containing a port number • transmits the message using the given port
`SendFile()`	• takes three parameters: ○ a string containing a text file name ○ a string containing a Destination ID ○ an integer containing a Port number • transmits the file one line at a time • transmits a final message with data string "`****`"

(a) Write pseudocode for module `SendFile()`.

Assume:

- module `Transmit()` has already been written and is used to transmit a message
- the value of `MyID` may be used as `SourceID`
- the file specified contains no blank lines
- the file specified does not contain the line "`****`"

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [7]

(b) Module `SendFile()` is used to copy a file from one computer to another.

A module within the program running on the destination computer will receive the data and write it to a new file.

Explain why module `SendFile()` transmits the message with data string `"****"` after the last line of the file.

.....

.....

.....

..... [2]

(c) One of the text files to be sent contains several blank lines (lines that do not contain any text).

(i) Explain why this is a problem.

.....

.....

.....

..... [2]

(ii) Explain how the message format could be changed to allow a blank line to be sent.

.....

.....

.....

..... [2]

Refer to the **insert** for the list of pseudocode functions and operators.

1 The following pseudocode represents part of the algorithm for a program.

Line numbers are for reference only.

```
10   DECLARE Sheet4 : ARRAY[1:2, 1:50] OF INTEGER
…
100  FOR PCount ← 0 TO 49
101     Sheet4[1, PCount] ← 0
102     Sheet4[2, PCount] ← 47
103  NEXT PCount
```

(a) The pseudocode contains references to an array.

Complete the table by writing the answer for each row.

	Answer
The dimension of the array	
The name of the variable used as an array index	
The number of elements in the array	

[3]

(c) The pseudocode does not include a declaration for `PCount`.

State the data type that should be used in the declaration.

..... [1]

3 A program processes data using a stack. The data is copied to a text file before the program ends.

(a) The following diagram shows the current state of the stack.

The operation of this stack may be summarised as follows:

- The `TopOfStack` pointer points to the last item added to the stack.
- The `BottomOfStack` pointer points to the first item on the stack.
- The stack grows upwards when items are added.

Stack		Pointer
Memory location	**Value**	
506		
505	WWW	← `TopOfStack`
504	YYY	
503	XXX	
502	ZZZ	
501	NNN	
500	PPP	← `BottomOfStack`

(i) An error will be generated if an attempt is made to POP a value when the stack is empty.

State the maximum number of consecutive POP operations that could be performed on the stack shown above **before** an error is generated.

..... [1]

(ii) The following operations are performed:

1. POP and store value in variable `Data1`
2. POP and store value in variable `Data2`
3. PUSH value AAA
4. PUSH value BBB
5. POP and discard value
6. POP and store value in variable `Data2`

Complete the diagram to show the state of the stack and the variables **after** the given operations have been performed.

Stack

Memory location	Value
506	
505	
504	
503	
502	
501	
500	

Pointer

Variable	Value
`Data1`	
`Data2`	

[4]

(b) The data is copied to a text file before the program ends.

(i) State an advantage of writing the data from the stack to a text file before the program ends.

.....

..... [1]

(ii) A module `SaveStack()` will write the data from the stack to a text file.

Express an algorithm for `SaveStack()` as five steps that could be used to produce pseudocode.

Write the **five** steps.

Step 1 .....

.....

Step 2 .....

.....

Step 3 .....

.....

Step 4 .....

.....

Step 5 .....

.....

[5]

8 A computer shop assembles desktop computers, using items bought from several suppliers. A text file `Stock.txt` contains information about each item.

Information for each item is stored as a single line in the `Stock.txt` file in the format:

```
<ItemNum><SupplierCode><Description>
```

Item information is as follows:

	Format	Comment
`ItemNum`	4 numeric characters	unique number for each item in the range “0001” to “5999” inclusive
`SupplierCode`	3 alphabetic characters	code to identify the supplier of the item
`Description`	a string	a minimum of 12 characters

The file is organised in ascending order of `ItemNum` and does not contain all possible values in the range.

The programmer has defined the first program module as follows:

Module	Description
`ChangeSupp()`	• called with two parameters `Code1` and `Code2` of type string that represent valid supplier codes • creates a new file `NewStock.txt` from the contents of the file `Stock.txt` where any reference to `Code1` is replaced by `Code2` • returns a count of the number of items that have had their supplier code changed

(a) Write pseudocode for module `ChangeSupp()`.

[8]

(b) A new module is required:

Module	Description

`Report_1()`	• takes a parameter of type string that represents a `SupplierCode`
	• searches the `Stock.txt` file for each line of item information that contains the given `SupplierCode`
	• produces a formatted report of items for the given `SupplierCode`, for example, for supplier DRG, the output could be:
	
	`Report for Supplier: DRG`
	
	`Item  Description`
	
	`1234  USB Printer Cable 3m`
	`1273  32GB USB Flash Drive`
	`1350  Mouse Mat 320 x 240mm`
	
	`Number of items listed: 3`

Write pseudocode for module `Report_1()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

(c) The format of the output from module `Report_1()` from **part (b)** is changed. The number of items listed is moved to the top of the report as shown in the example:

```
Report for Supplier: DRG
Number of items listed: 3

Item        Description

1234        USB Printer Cable 3m
1273        32GB USB Flash Drive
1350        Mouse Mat 320 x 240mm
```

(i) Explain why this new layout would increase the complexity of the algorithm.

.....

.....

.....

..... [2]

(ii) The algorithm will be modified to produce the report in the new format. The modified algorithm will be implemented so that the file `Stock.txt` is only read once.

Describe the modified algorithm.

.....

.....

.....

.....

.....

.....

..... [3]

Refer to the **insert** for the list of pseudocode functions and operators.

1 A program calculates the postal cost based on the weight of the item and its destination. Calculations occur at various points in the program and these result in the choice of several possible postal costs. The programmer has built these postal costs into the program.

For example, the postal cost of $3.75 is used in the following lines of pseudocode:

```
IF Weight < 250 AND ValidAddress = TRUE THEN
   ItemPostalCost ← 3.75    // set postal cost for item to $3.75
   ItemStatus ← "Valid"     // item can be sent
ENDIF
```

(a) (i) Identify a more appropriate way of representing the postal costs.

..... [1]

(c) Give the **appropriate** data types for the following variables:

`ValidAddress` .....

`ItemPostalCost` .....

`ItemStatus` .....

[3]

2 A program stores a user's date of birth using a variable `MyDOB` of type `DATE`.

(a) Write a pseudocode statement, using a function from the **insert**, to assign the value corresponding to 17/11/2007 to `MyDOB`.

..... [1]

(c) The program will output the day of the week corresponding to `MyDOB`.

For example, given the date 22/06/2023, the program will output `"Thursday"`.

An algorithm is required. An array will be used to store the names of the days of the week.

Define the array **and** describe the algorithm in **four** steps.

Do **not** use pseudocode statements in your answer.

Array definition .....

.....

Step 1 .....

.....

.....

Step 2 .....

.....

.....

Step 3 .....

.....

.....

Step 4 .....

.....

.....

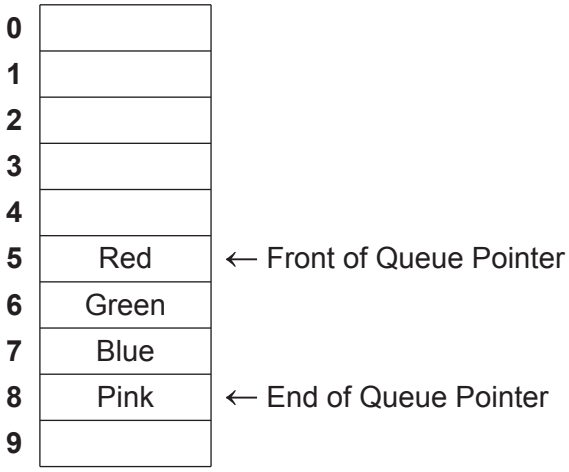
[6]

3 A program stores data in a text file. When data is read from the file, it is placed in a queue.

(a) The diagram below represents an Abstract Data Type (ADT) implementation of the queue. Each data item is stored in a separate location in the data structure. During initial design, the queue is limited to holding a maximum of 10 data items.

The operation of this queue may be summarised as follows:

- The Front of Queue Pointer points to the next data item to be removed.
- The End of Queue Pointer points to the last data item added.
- The queue is circular so that locations can be reused.



(i) Describe how the data items Orange and Yellow are added to the queue shown in the diagram.

.....

.....

.....

.....

.....

.....

.....

..... [4]

(ii) The following diagram shows the state of the queue after several operations have been performed. All queue locations have been used at least once.

0	D4	
1	D3	← End of Queue Pointer
2	D27	
3	D8	
4	D33	
5	D17	← Front of Queue Pointer
6	D2	
7	D1	
8	D45	
9	D60	

State the number of data items in the queue.

..... [1]

(b) The design of the queue is completed and the number of locations is increased.

A function `AddToQueue()` has been written. It takes a string as a parameter and adds this to the queue. The function will return `TRUE` if the string was added successfully.

A procedure `FileToQueue()` will add each line from the file to the queue. This procedure will end when all lines have been added or when the queue is full.

Describe the algorithm for procedure `FileToQueue()`.

Do **not** use pseudocode in your answer.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [5]

8 A computer shop assembles computers using items bought from several suppliers. A text file `Stock.txt` contains information about each item.

Information for each item is stored as a single line in the `Stock.txt` file in the format:

```
<ItemNum><SupplierCode><Description>
```

Valid item information is as follows:

	Format	Comment
`ItemNum`	4 numeric characters	unique number for each item in the range ″0001″ to ″5999″ inclusive
`SupplierCode`	3 alphabetic characters	to identify the supplier of the item
`Description`	a string	a minimum of 12 characters

The file is organised in ascending order of `ItemNum` and does **not** contain all possible values in the range.

A programmer has started to define program modules as follows:

Module	Description
`OnlyAlpha()` (already written)	• called with a parameter of type string • returns `TRUE` if the string contains only alphabetic characters, otherwise returns `FALSE`
`CheckInfo()`	• called with a parameter of type string representing a line of item information • checks to see whether the item information in the string is valid • returns `TRUE` if the item information is valid, otherwise returns `FALSE`

(a) Write pseudocode for module `CheckInfo()`.

Module `OnlyAlpha()` should be used as part of your solution.

.....

.....

.....

..... [7]

(b) A new module is defined as follows:

Module	Description
`AddItem()`	• called with a parameter of type string representing valid information for a new item that is not currently in the `Stock.txt` file • creates a new file `NewStock.txt` from the contents of the file `Stock.txt` and adds the new item information at the appropriate place in the `NewStock.txt` file

As a reminder, the file `Stock.txt` is organised in ascending order of `ItemNum` and does not contain all possible values in the range.

Write pseudocode for module `AddItem()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

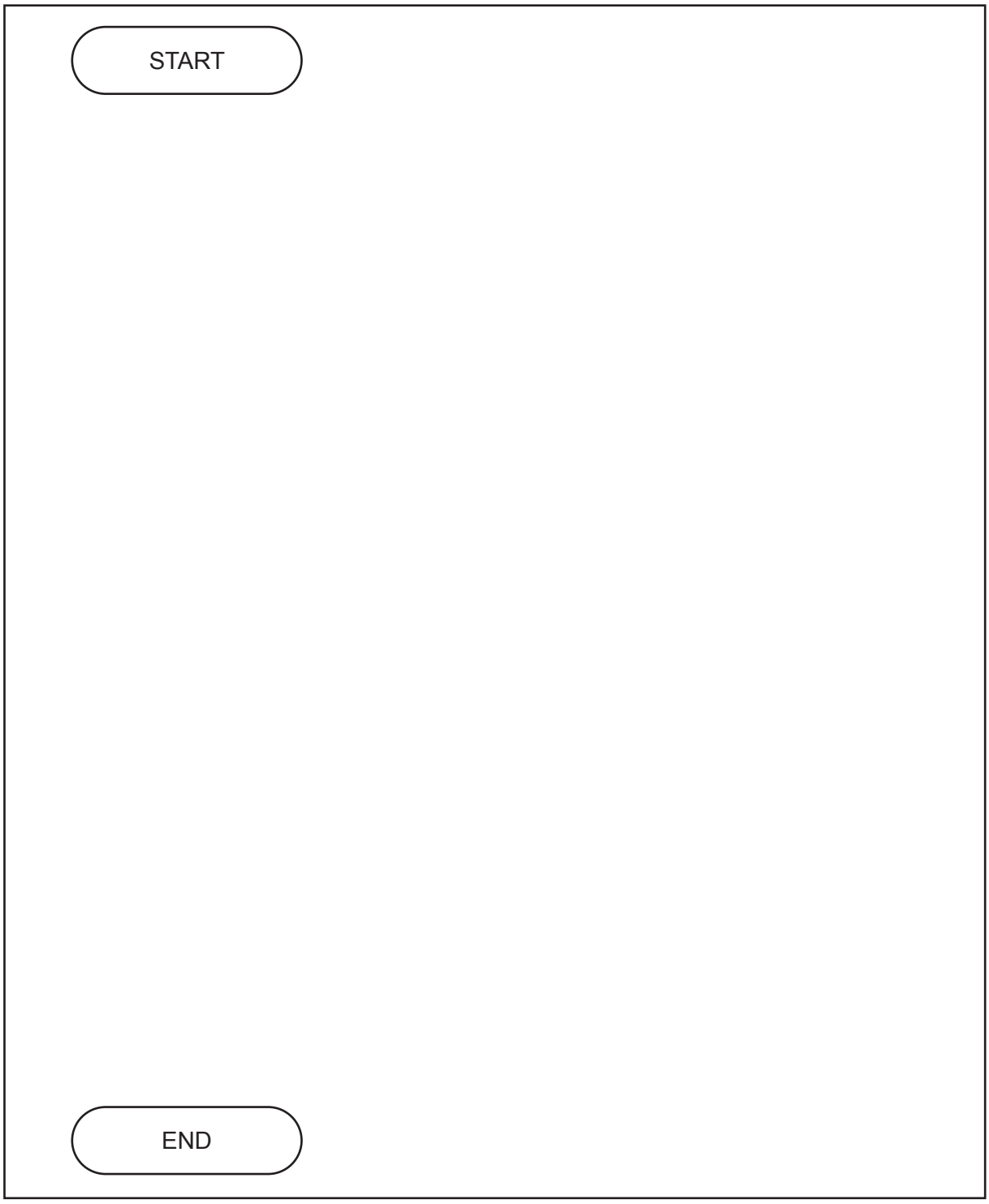
..... [7]

2 A program contains an algorithm to output a string of a specified length containing identical characters.

(a) The algorithm is described as follows:

1. prompt and input a character and store in `MyChar`
2. prompt and input an integer and store in `MyCount`
3. generate a string consisting of `MyChar` repeated `MyCount` times
4. output the string.

Draw a program flowchart to represent the algorithm.



[4]

(b) A different part of the program uses the variable `StartDate`.

Write pseudocode statements to declare `StartDate` **and** assign to it the date corresponding to 15/11/2005.

Declaration .....

Assignment .....

[3]

3 Customers collect points every time they make a purchase at a store.

A program is used to manage the points system and the table lists some of the information stored for one customer.

Information	Data type required
Name	String
Number of points collected	Integer
Date of birth	Date

(a) (i) Identify a suitable structure for storing the information for one customer. Explain the advantage of using this structure.

Structure .....

Advantage .....

.....

.....

.....

.....

..... [4]

(ii) Describe a data structure that could be used to store the information for **all** customers.

.....

..... [2]

6 A video-conferencing program supports up to six users. Speech from each user is sampled and digitised (converted from analogue to digital). Digitised values are stored in array `Sample`.

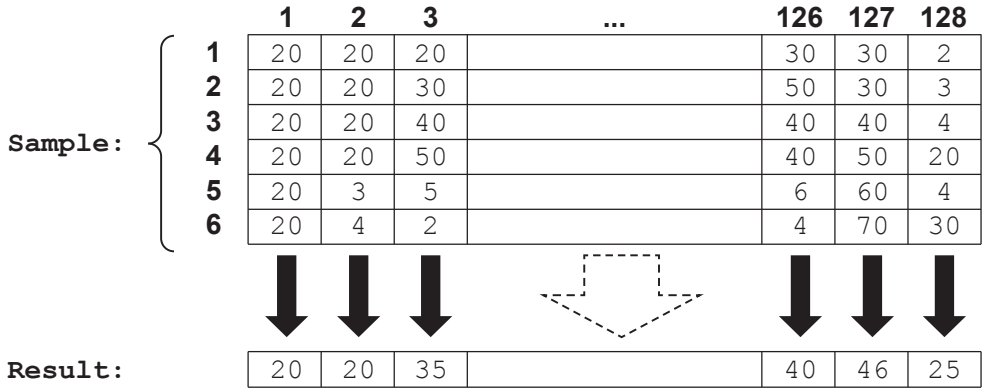
The array `Sample` consists of 6 rows by 128 columns and is of type integer. Each row contains 128 digitised sound samples from one user.

The digitised sound samples from each user are to be processed to produce a single value which will be stored in a 1D array `Result` of type integer. This process will be implemented by procedure `Mix()`.

A procedure `Mix()` will:

- calculate the average of each of the 6 sound samples in a column
- ignore sound sample values of 10 or less
- store the average value in the corresponding position in `Result`
- repeat for each column in array `Sample`

The diagram uses example values to illustrate the process:



Write pseudocode for procedure `Mix()`.

Assume `Sample` and `Result` are global.

..... [6]

8 A computer shop assembles computers using items bought from several suppliers. A text file `Stock.txt` contains information about each item.

Information for each item is stored as a single line in the `Stock.txt` file in the format:

```
<ItemNum><SupplierCode><Description>
```

Item information is as follows:

	Format	Comment
`ItemNum`	4 numeric characters	unique for each item in the range "0001" to "5999" inclusive
`SupplierCode`	5 alphabetic characters	to identify the supplier of the item
`Description`	a string	a minimum of 12 characters

The file is organised in ascending order of `ItemNum` and does **not** contain all possible values in the range.

A programmer has started to define program modules as follows:

Module	Description
`SuppExists()` (already written)	• called with a parameter of type string representing a supplier code • returns `TRUE` if the supplier code is already in use, otherwise returns `FALSE`
`IsNewSupp()`	• called with a parameter of type string representing a new supplier code • returns `TRUE` if the string only contains alphabetic characters (either upper or lower case) and the supplier code is **not** already in use, otherwise returns `FALSE`

(a) Write pseudocode for module `IsNewSupp()`.

Module `SuppExists()` has already been written and should be used as part of your solution. Module `SuppExists()` will generate a run-time error if the given parameter is not 5 characters in length.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [7]

(b) A new module has been defined:

Module	Description
`CheckNewItem()`	• called with a parameter of type string representing a line of item information • checks to see whether an item with the same `ItemNum` already exists in the file • returns `TRUE` if the `ItemNum` is not already in the file, otherwise returns `FALSE`

Write **efficient** pseudocode for module `CheckNewItem()`.

.....

.....

.....

..... [7]

(d) A new module `AddItem()` will be used to add information to the `Stock.txt` file.

State the file mode that should be used for the algorithm within this module.

..... [1]

(e) A new module `FindItem()` searches for a given item in the `Stock.txt` file, which is already organised in ascending order of `ItemNum`.

Describe how this organisation may improve the efficiency of the algorithm.

.....

.....

.....

.....

.....

..... [3]

Refer to the **insert** for the list of pseudocode functions and operators.

1 A program is required for a shopping website.

(a) Part of the program requires four variables. The following table describes the use of each variable.

Complete the table by adding the most appropriate data type for each variable.

Variable use	Data type
Store the number of days in the current month	
Store the first letter of the customer's first name	
Store an indication of whether a year is a leap year	
Store the average amount spent per customer visit	

[4]

4 **(a)** A program contains a 1D array `DataItem` with 100 elements.

State the **one additional** piece of information required before the array can be declared.

.....

..... [1]

(b) A programmer decides to implement a queue Abstract Data Type (ADT) in order to store characters received from the keyboard. The queue will need to store at least 10 characters and will be implemented using an array.

(i) Describe **two** operations that are typically required when implementing a queue. State the check that must be carried out before each operation can be completed.

Operation 1 .....

.....

Check 1 .....

.....

Operation 2 .....

.....

Check 2 .....

.....

[4]

(ii) Describe the declaration and initialisation of the variables and data structures used to implement the queue.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [5]

5 **(a)** A text string contains three data items concatenated as shown:

<StockID><Description><Cost>

Item lengths are:

Item	Length
StockID	5
Description	32
Cost	the remainder of the string

A procedure `Unpack()` takes four parameters of type string. One parameter is the original text string. The other three parameters are used to represent the three data items shown in the table and are assigned values within the procedure. These values will be used by the calling program after the procedure ends.

(i) Write pseudocode for the procedure `Unpack()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

(b) The design changes and a record structure is defined to store the three data items.

A user-defined data type `StockItem` is created as shown:

```
TYPE StockItem
   DECLARE StockID : STRING
   DECLARE Description : STRING
   DECLARE Cost : REAL
ENDTYPE
```

(i) A variable `LineData` of type `StockItem` is declared.

Write the pseudocode statement to assign the value `12.99` to the `Cost` field of `LineData`.

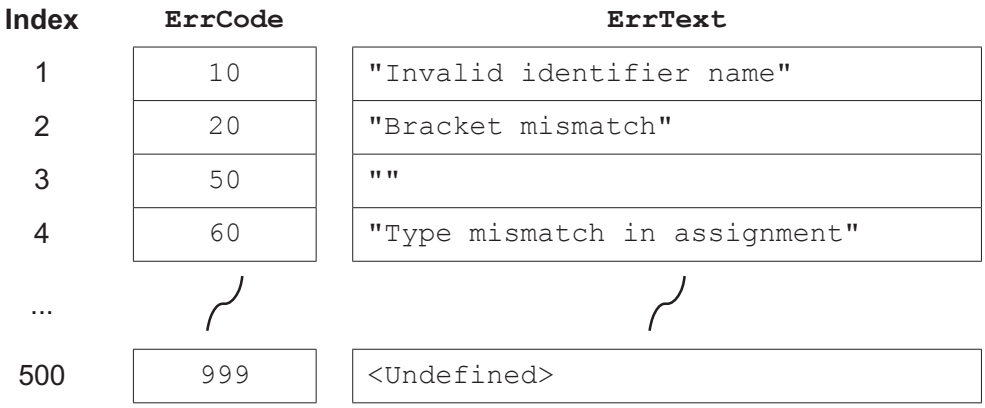
..... [1]

7 A teacher is designing a program to perform simple syntax checks on programs written by students.

Two global 1D arrays are used to store the syntax error data. Both arrays contain 500 elements.

- Array `ErrCode` contains integer values that represent an error number in the range 1 to 800.
- Array `ErrText` contains string values that represent an error description.

The following diagram shows an example of the arrays.



Note:

- There are less than 500 error codes so corresponding elements in both arrays may be unused. Unused elements in `ErrCode` have the value 999. These will occur at the end of the array. The value of unused elements in `ErrText` is undefined.
- Values in the `ErrCode` array are stored in ascending order but not all values may be present. For example, there may be no error code 31.
- Some error numbers are undefined. In these instances, the `ErrCode` array will contain a valid error number but the corresponding `ErrText` element will contain an empty string.

The teacher has defined one program module as follows:

Module	Description

`OutputRange()`	• Prompts for input of two error numbers
	• Outputs a list of error numbers between the two numbers input (inclusive) together with the corresponding error description
	• Outputs a warning message when the error description is missing as for error number 50 in the example
	• Outputs a suitable header and a final count of error numbers found
	
	Output based on the example array data above:
	
	`List of error numbers from 1 to 60`
	
	`10 : Invalid identifier name`
	`20 : Bracket mismatch`
	`50 : Error Text Missing`
	`60 : Type mismatch in assignment`
	
	`4 error numbers output`

(a) Write pseudocode for module `OutputRange()`. Assume that the two numbers input represent a valid error number range.

.....

.....

.....

.....

.....

.....

.....

.....

..... [8]

Question 7 continues on the next page.

(b) (i) Two additional modules are defined:

Module	Description
`SortArrays()`	• Sorts the arrays into ascending order of `ErrCode`
`AddError()`	• Takes two parameters: ○ an error number as an integer ○ an error description as a string • Writes the error number and error description to the first unused element of the two arrays. Ensures the `ErrCode` array is still in ascending order • Returns the number of unused elements **after** the new error number has been added • Returns −1 if the new error number could not be added

Write pseudocode for the module `AddError()`. Assume that the error code is **not** already in the `ErrCode` array.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

(ii) A new Module `RemoveError()` will remove a given error number from the array.

Describe the algorithm that would be required. Do **not** include pseudocode statements in your answer.

.....

.....

.....

.....

.....

.....

..... [3]

2 **(a)** An algorithm will process data from a test taken by a group of students. The algorithm will prompt and input the name and test mark for each of the 35 students.

The algorithm will add the names of all the students with a test mark of less than 20 to an existing text file `Support_List.txt`, which already contains data from other group tests.

(i) Describe the steps that the algorithm should perform.

Do **not** include pseudocode statements in your answer.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [5]

(ii) Explain why it may be better to store the names of the students in a file rather than in an array.

.....

.....

..... [1]

(iii) Explain why `WRITE` mode cannot be used in the answer to **part 2(a)(i)**.

.....

.....

..... [1]

3 A stack is used in a program to store string data which needs to be accessed in several modules.

(a) A stack is an example of an Abstract Data Type (ADT).

Identify **one other** example of an ADT **and** describe its main features.

Example .....

Features .....

.....

.....

.....

.....

.....

[3]

(b) Explain how the stack can be implemented using an array.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

[5]

(c) A second stack is used in the program. The diagram below shows the initial state of this stack. Value X is at the top of the stack and was the last item added.

Upper-case letters are used to represent different data values.

Stack operations are performed in three groups as follows:

Group 1:

```
PUSH D
PUSH E
```

Group 2:

```
POP
POP
POP
```

Group 3:

```
PUSH A
PUSH B
POP
PUSH C
```

Complete the diagram to show the state of the stack **after** each group of operations has been performed.

Include the current stack pointer (SP) **after** each group.

Memory location	Initial state		After Group 1	After Group 2	After Group 3
957					
956					
955					
954					
953	X	←SP			
952	Y				
951	Z				
950	P				

[5]

7 A teacher is designing a program to perform simple syntax checks on programs written by students.

Two global 1D arrays are used to store the syntax error data. Both arrays contain 500 elements.

- Array `ErrCode` contains integer values that represent an error number in the range 1 to 800.
- Array `ErrText` contains string values that represent an error description.

The following diagram shows an example of the arrays.

Index	ErrCode	ErrText
1	10	"Invalid identifier name"
2	20	"Bracket mismatch"
3	50	"Undeclared variable"
4	60	"Type mismatch in assignment"
…		
500	999	<Undefined>

Note:

- There may be less than 500 error numbers so corresponding elements in both arrays may be unused. Unused elements in `ErrCode` have the value `999`. The value of unused elements in `ErrText` is undefined.
- Values in the `ErrCode` array are stored in ascending order but not all values may be present, for example, there may be no error code `31`.

The teacher has defined two program modules as follows:

Module	Description

`OutputError()`	• takes two parameters as integers:
	  ○ a line number in the student's program
	  ○ an error number
	• searches for the error number in the `ErrCode` array:
	  ○ if found, outputs the corresponding error description and the line number, for example:
	  `"Bracket mismatch on line 34"`
	  ○ if not found, outputs the line number and a warning, for example:
	  `"Unknown error on line 34"`

`SortArrays()`	sorts the arrays into ascending order of `ErrCode`

(a) Write **efficient** pseudocode for module `OutputError()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

(b) Write an **efficient** bubble sort algorithm in pseudocode for module `SortArrays()`.

..... [8]

(c) Two 1D arrays were described at the beginning of the question. Both arrays contain 500 elements.

- Array `ErrCode` contains integer values that represent an error number in the range 1 to 800.
- Array `ErrText` contains string values that represent an error description.

The two arrays will be replaced by a single array. A user-defined data type (record structure) has been declared as follows:

```
TYPE ErrorRec
   DECLARE ErrCode : STRING
   DECLARE ErrText : STRING
ENDTYPE
```

(i) State the error in the record declaration.

.....

..... [1]

(ii) State **two** benefits of using the single array of the user-defined data type.

1 .....

.....

2 .....

.....

[2]

(iii) Write the declaration for the single array in pseudocode.

..... [1]

Refer to the **insert** for the list of pseudocode functions and operators.

1 **(a)** An algorithm includes a number of complex calculations. A programmer is writing a program to implement the algorithm and decides to use library routines to provide part of the solution.

State **three** possible benefits of using library routines in the development of the program.

1 .....

.....

2 .....

.....

3 .....

.....

[3]

(b) The following pseudocode is part of a program that stores names and test marks for use in other parts of the program.

```
DECLARE Name1, Name2, Name3 : STRING
DECLARE Mark1, Mark2, Mark3 : INTEGER
INPUT Name1
INPUT Mark1
INPUT Name2
INPUT Mark2
INPUT Name3
INPUT Mark3
```

(i) The pseudocode needs to be changed to allow for data to be stored for up to 30 students.

Explain why it would be good practice to use arrays to store the data.

.....

.....

.....

.....

.....

..... [3]

(ii) The following pseudocode statement includes array references:

```
OUTPUT "Student ", Name[Count], " scored ", Mark[Count]
```

State the purpose of the variable `Count` and give its data type.

Purpose .....

.....

Data type .....

[2]

3 A 1D array `Data` of type integer contains 200 elements. Each element has a unique value.

An algorithm is required to search for the largest value and output it.

Describe the steps that the algorithm should perform.

Do **not** include pseudocode statements in your answer.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

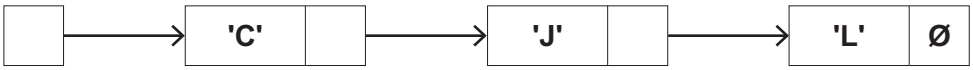
.....

..... [5]

4 **(a)** The following diagram shows an Abstract Data Type (ADT) representation of an ordered linked list. The data item stored in each node is a single character. The data will be accessed in alphabetical order.

The symbol Ø represents a null pointer.

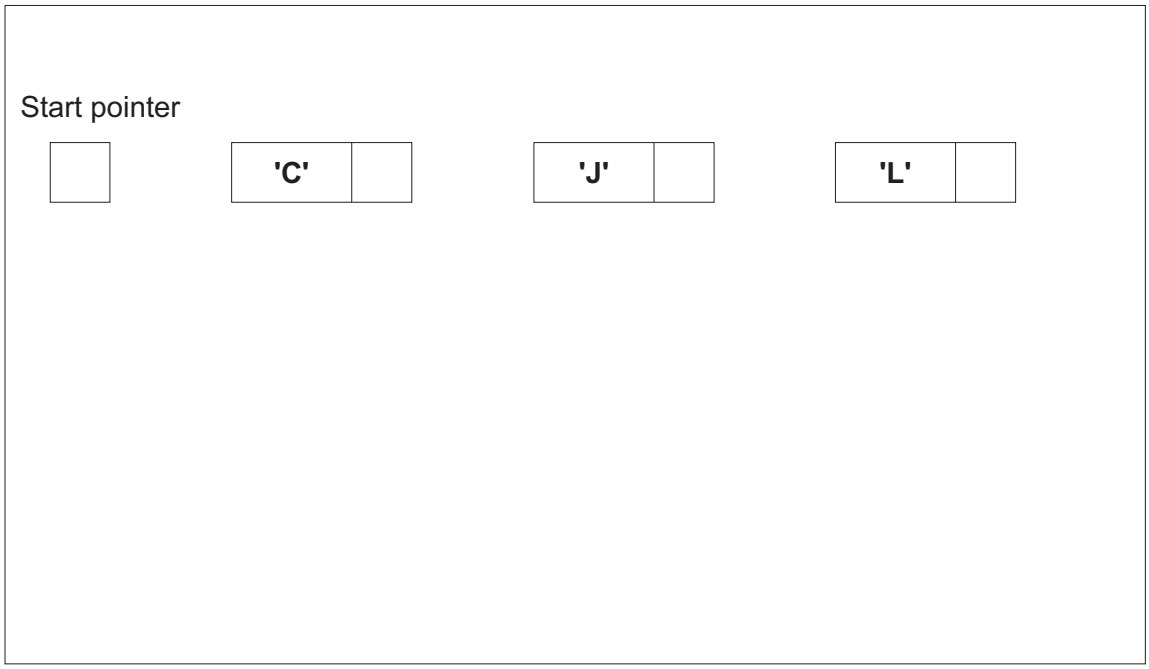
Start pointer



(i) Nodes with data 'A' and 'K' are added to the linked list. Nodes with data 'J' and 'L' are deleted.

After the changes, the data items still need to be accessed in alphabetical order.

Complete the diagram to show the new state of the linked list.



[4]

(ii) The original data could have been stored in a 1D array in which each element stores a character.

For example:

'C'	'J'	'L'		

Explain the advantages of making the changes described in **part (a)(i)** when the data is stored in the linked list instead of an array.

.....

.....

.....

..... [2]

(iii) Explain the disadvantages of making the changes described in **part (a)(i)** when the data is stored in the linked list instead of an array.

.....

.....

.....

..... [2]

(b) A program will store data using a linked list like the one shown in **part (a)**.

Explain how the linked list can be implemented.

.....

.....

.....

.....

.....

.....

.....

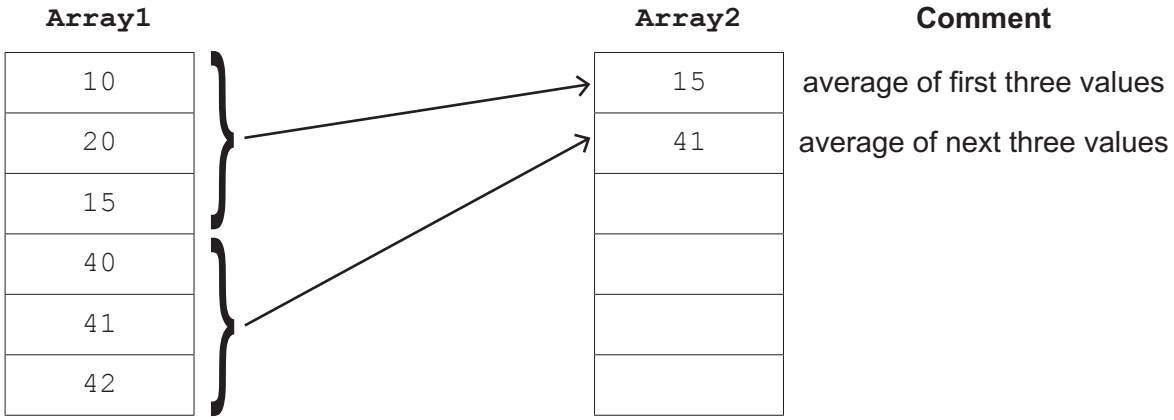
..... [4]

5 A program uses two 1D arrays of type integer. `Array1` contains 600 elements and `Array2` contains 200 elements.

`Array1` contains sample values read from a sensor. The sensor always takes three consecutive samples and all of these values are stored in `Array1`.

A procedure `Summarise()` will calculate the average of three consecutive values from `Array1` and write the result to `Array2`. This will be repeated for all values in `Array1`.

The diagram below illustrates the process for the first six entries in `Array1`.



Write pseudocode for the procedure `Summarise()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [5]

8 A teacher is designing a program to perform simple syntax checks on programs written by students. Student programs are submitted as text files, which are known as project files.

A project file may contain blank lines.

The teacher has defined the first program module as follows:

Module	Description
`CheckFile()`	• takes the name of an existing project file as a parameter of type string • returns `TRUE` if the file is valid (it contains at least 10 non-blank lines), otherwise returns `FALSE`

(a) Write pseudocode for module `CheckFile()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [7]

Further modules are defined as follows:

Module	Description
`CheckLine()`	• takes a line from a project file as a parameter of type string • returns zero if the line is blank or contains no syntax error, otherwise returns an error number as an integer
`CountErrors()`	• takes two parameters: ○ the name of a project file as a string ○ the maximum number of errors as an integer • uses `CheckFile()` to test the project file. Outputs an error message and ends if the project file is not valid • calls `CheckLine()` for each line in the project file • counts the number of errors • outputs the number of errors or a warning message if the maximum number of errors is exceeded

(b) `CountErrors()` is called to check the project file `Jim01Prog.txt` and to stop if more than 20 errors are found.

Write the pseudocode statement for this call.

.....

..... [2]

(c) Write pseudocode for module `CountErrors()`. Assume `CheckFile()` and `CheckLine()` have been written and can be used in your solution.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [8]

Refer to the **insert** for the list of pseudocode functions and operators.

1 Sylvia is testing a program that has been written by her colleague. Her colleague tells her that the program does not contain any syntax errors.

(a) (i) State what her colleague means by “does not contain any syntax errors”.

.....

.....

..... [1]

(b) Complete the following table by giving the appropriate data type in each case.

Use of variable	Data type
The average mark in a class of 40 students	
An email address	
The number of students in the class	
To indicate whether an email has been read	

[4]

2 **(a)** An algorithm to sort a 1D array into ascending order is described as follows:

- move the largest value to the end
- keep repeating until the array is sorted.

Apply the process of stepwise refinement to this algorithm in order to produce a more detailed description.

Write the more detailed description using **structured English**. Your explanation of the algorithm should not include pseudocode statements.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

3 **(a)** The diagram below represents a queue Abstract Data Type (ADT) that can hold a maximum of eight items.

The operation of this queue may be summarised as follows:

- The front of queue pointer points to the next item to be removed.
- The end of queue pointer points to the last item added.
- The queue is circular so that empty storage elements can be reused.

0	Frog	← Front of queue pointer
1	Cat	
2	Fish	
3	Elk	← End of queue pointer
4		
5		
6		
7		

(i) Describe how "Octopus" is added to the given queue.

.....

.....

.....

..... [2]

(ii) Describe how the next item in the given queue is removed and stored in the variable `AnimalName`.

.....

.....

.....

..... [2]

(iii) Describe the state of the queue when the **front of queue** and the **end of queue** pointers have the same value.

.....

..... [1]

(b) Some operations are carried out on the original queue given in **part (a)**.

(i) The current state of the queue is:

0	Frog
1	Cat
2	Fish
3	Elk
4	
5	
6	
7	

Complete the diagram to show the state of the queue after the following operations:

Add “Wasp”, “Bee” and “Mouse”, and then remove two data items.

[3]

(ii) The state of the queue after other operations are carried out is shown:

0	Frog	
1	Cat	
2	Fish	
3	Elk	← Front of queue pointer
4	Wasp	
5	Bee	
6	Mouse	← End of queue pointer
7	Ant	

Complete the following diagram to show the state of the queue after the following operations:

Remove one item, and then add “Dolphin” and “Shark”.

0	
1	
2	
3	
4	
5	
6	
7	

[2]

(c) The queue is implemented using a 1D array.

Describe the algorithm that should be used to modify the **end of queue pointer** when adding an item to the queue.

Your algorithm should detect any potential error conditions.

.....

.....

.....

.....

.....

..... [3]

5 The following data items will be recorded each time a student successfully logs on to the school network:

Data item	Example data
Student ID	`"CJL404"`
Host ID	`"Lib01"`
Time and date	`"08:30, June 01, 2021"`

The Student ID is six characters long. The other two data items are of variable length.

A single string will be formed by concatenating the three data items. A separator character will need to be inserted between items two and three.

For example:

```
"CJL404Lib01<separator>08:30, June 01, 2021"
```

Each string represents one log entry.

A programmer decides to store the concatenated strings in a 1D array `LogArray` that contains 2000 elements. Unused array elements will contain an empty string.

(a) Suggest a suitable separator character **and** give a reason for your choice.

Character .....

Reason .....

.....

[2]

(c) A function `LogEvents()` will:

- take a Student ID as a parameter
- for each element in the array that matches the Student ID parameter:
  - add the value of the array element to the existing text file `LogFile`
  - assign an empty string to the array element
- count the number of lines added to the file
- return this count.

Write pseudocode for the function `LogEvents()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

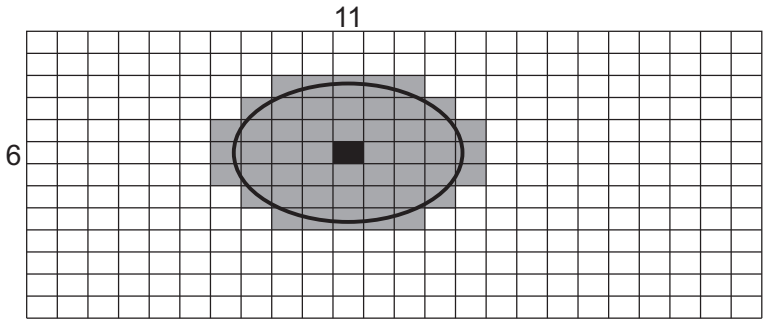
..... [7]

6 A mobile phone has a touchscreen. The screen is represented by a grid, divided into 800 rows and 1280 columns.

The grid is represented by a 2D array `Screen` of type `INTEGER`. An array element will be set to 0 unless the user touches that part of the screen.

Many array elements are set to 1 by a single touch of a finger or a stylus.

The following diagram shows a simplified touchscreen. The dark line represents a touch to the screen. All grid elements that are wholly or partly inside the outline will be set to 1. These elements are shaded.
The element shaded in black represents the centre point.



A program is needed to find the coordinates (the row and column) of the centre point. The centre point on the diagram above is row 6, column 11.

Assume:

- the user may only touch one area at a time
- screen rotation does not affect the touchscreen.

The programmer has started to define program modules as follows:

Module	Description
`SetRow()` (generates test data)	• Called with three parameters of type `INTEGER`: ◦ a row number ◦ the number of pixels to be skipped starting from column 1 ◦ the number of pixels that should be set to 1 • Sets the required number of pixels to 1 For example, `SetRow(3, 8, 5)` will give row 3 as in the diagram shown.
`SearchInRow()`	• Takes two parameters of type `INTEGER`: ◦ a row number ◦ a start column (1 or 1280) • Searches the given row from the start column (either left to right or right to left) for the first column that contains an element set to 1 • Returns the column number of the first element in the given row that is set to 1 • Returns −1 if no element is set to 1
`SearchInCol()`	• Takes two parameters of type `INTEGER`: ◦ a column number ◦ a start row (1 or 800) • Searches the given column from the start row (either up or down) for the first row that contains an element set to 1 • Returns the row number of the first element in the given column that is set to 1 • Returns −1 if no element is set to 1

(a) Write pseudocode to implement the module `SetRow()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [5]

(b) The module description of `SearchInRow()` is provided here for reference.

Module	Description
`SearchInRow()`	• Takes two parameters of type `INTEGER`: ◦ a row number ◦ a start column (1 or 1280) • Searches the given row from the start column (either left to right or right to left) for the first column that contains an element set to 1 • Returns the column number of the first element in the given row that is set to 1 • Returns −1 if no element is set to 1

Write pseudocode to implement the module `SearchInRow()`.

.....

.....

.....

.....

.....

.....

.....

..... [8]

Refer to the **insert** for the list of pseudocode functions and operators.

1 **(a)** A programmer applies decomposition to a problem that she has been asked to solve.

Describe decomposition.

.....

.....

.....

.....

.....

..... [2]

(b) The following pseudocode assigns a value to an element of an array:

```
ThisArray[n] ← 42
```

Complete the following table by writing the answer for each row.

	Answer
The number of dimensions of `ThisArray`	
The technical terms for minimum and maximum values that the variable `n` may take	
The technical term for the variable `n` in the pseudocode statement	

[3]

3 A programmer is writing a program to help manage clubs in a school.

Data will be stored about each student in the school and each student may join up to three clubs.

The data will be held in a record structure of type `Student`.

The programmer has started to define the fields that will be needed as shown in the following table.

Field	Typical value	Comment
`StudentID`	`"CF1234"`	Unique to each student
`Email`	`"Carmen47@xyzmail.com"`	Contains letters, numbers and certain symbols
`Club_1`	`1`	Any value in the range 1 to 99 inclusive
`Club_2`	`14`	Any value in the range 1 to 99 inclusive
`Club_3`	`27`	Any value in the range 1 to 99 inclusive

(a) (i) Write pseudocode to declare the record structure for type `Student`.

.....

.....

.....

.....

.....

.....

..... [3]

(ii) A 1D array `Membership` containing 3000 elements will be used to store the student data.

Write pseudocode to declare the `Membership` array.

.....

..... [2]

(iii) Some of the elements of the array will be unused.

Give an **appropriate** way of indicating an unused array element.

.....

..... [1]

(iv) Some students are members of less than three clubs.

State **one** way of indicating an unused club field.

.....

..... [1]

(b) A procedure `GetIDs()` will:

- prompt and input the number of a club
- output the `StudentID` of all the students who are members of that club
- output a count of all students in the given club.

Write pseudocode for the procedure `GetIDs()`.

.....

.....

.....

5 A company has several departments. Each department stores the name, email address and the status of each employee in that department in its own text file. All text files have the same format.

Employee details are stored as three separate data strings on three consecutive lines of the file. An example of the first six lines of one of the files is as follows:

File line	Comment
1	First employee name
2	First email address
3	First employee status
4	Second employee name
5	Second email address
6	Second employee status

A procedure `MakeNewFile()` will:

- take three parameters as strings:
  - an existing file name
  - a new file name
  - a search status value
- create a new text file using the new file name
- write all employee details to the new file where the employee status is **not** equal to the search status value
- count the number of sets of employee details that were in the original file
- count the number of sets of employee details that were written to the new file
- produce a summary output.

An example summary output is as follows:

```
File Marketing contained 54 employee details
52 employee sets of details were written to file NewMarketingList
```

(a) Write pseudocode for the procedure `MakeNewFile()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

(ii) State **one** advantage **and one** disadvantage of the alternative format.

Advantage .....

.....

Disadvantage .....

.....

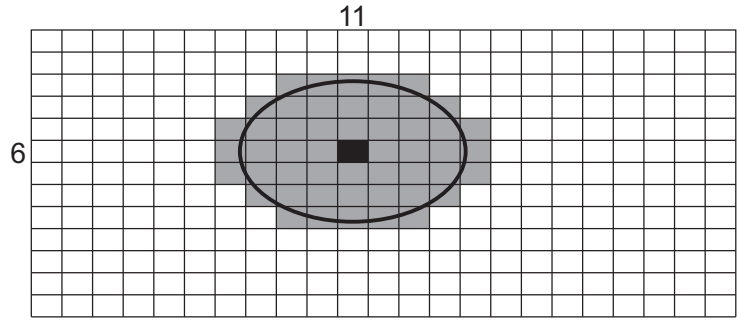
[2]

6 A mobile phone has a touchscreen. The screen is represented by a grid, divided into 800 rows and 1280 columns.

The grid is represented by a 2D array `Screen` of type `INTEGER`. An array element will be set to 0 unless the user touches that part of the screen.

Many array elements will set to 1 by a single touch of a finger or a stylus.

The following diagram shows a simplified touchscreen. The dark line represents a touch on the screen. All grid elements that are wholly or partly inside the outline will be set to 1. These elements are shaded.
The element shaded in black represents the centre point of the touch.



A program is needed to find the coordinates (the row and column) of the centre point. The centre point on the diagram shown is row 6, column 11.

Assume:

- the user may only touch one area at a time
- screen rotation does not affect the touchscreen.

The programmer has decided to use global values `CentreRow` and `CentreCol` as coordinate values for the centre point.

The programmer has started to define program modules as follows:

Module	Description
`FirstRowSet()`	• Searches for the first row that has an array element set to 1 • Returns the index of that row (1 is the first row) • Returns −1 if there are no elements set to 1
`LastRowSet()`	• Searches for the last row that has an array element set to 1 • Returns the index of that row • Returns −1 if there are no elements set to 1
`FirstColSet()`	• Searches for the first column that has an array element set to 1 • Returns the index of that column (1 is the first column) • Returns −1 if there are no elements set to 1
`LastColSet()`	• Searches for the last column that has an array element set to 1 • Returns the index of that column • Returns −1 if there are no elements set to 1

(a) Write efficient pseudocode for the module `FirstRowSet()`.

[7]

(b) Describe a feature of your solution to **part (a)** that indicates the pseudocode represents an efficient algorithm.

.....

.....

.....

..... [2]

Refer to the **insert** for the list of pseudocode functions and operators.

1 Sylvia is testing a program that has been written by her colleague. Her colleague tells her that the program does not contain any syntax errors.

(a) (i) State what her colleague means by “does not contain any syntax errors”.

.....

.....

..... [1]

(b) Complete the following table by giving the appropriate data type in each case.

Use of variable	Data type
The average mark in a class of 40 students	
An email address	
The number of students in the class	
To indicate whether an email has been read	

[4]

3 **(a)** The diagram below represents a queue Abstract Data Type (ADT) that can hold a maximum of eight items.

The operation of this queue may be summarised as follows:

- The front of queue pointer points to the next item to be removed.
- The end of queue pointer points to the last item added.
- The queue is circular so that empty storage elements can be reused.

0	Frog	← Front of queue pointer
1	Cat	
2	Fish	
3	Elk	← End of queue pointer
4		
5		
6		
7		

(i) Describe how "Octopus" is added to the given queue.

.....

.....

.....

..... [2]

(ii) Describe how the next item in the given queue is removed and stored in the variable `AnimalName`.

.....

.....

.....

..... [2]

(iii) Describe the state of the queue when the **front of queue** and the **end of queue** pointers have the same value.

.....

..... [1]

(b) Some operations are carried out on the original queue given in **part (a)**.

(i) The current state of the queue is:

0	Frog
1	Cat
2	Fish
3	Elk
4	
5	
6	
7	

Complete the diagram to show the state of the queue after the following operations:

Add “Wasp”, “Bee” and “Mouse”, and then remove two data items.

[3]

(ii) The state of the queue after other operations are carried out is shown:

0	Frog	
1	Cat	
2	Fish	
3	Elk	← Front of queue pointer
4	Wasp	
5	Bee	
6	Mouse	← End of queue pointer
7	Ant	

Complete the following diagram to show the state of the queue after the following operations:

Remove one item, and then add “Dolphin” and “Shark”.

0	
1	
2	
3	
4	
5	
6	
7	

[2]

(c) The queue is implemented using a 1D array.

Describe the algorithm that should be used to modify the **end of queue pointer** when adding an item to the queue.

Your algorithm should detect any potential error conditions.

.....

.....

.....

.....

.....

..... [3]

5 The following data items will be recorded each time a student successfully logs on to the school network:

Data item	Example data
Student ID	`"CJL404"`
Host ID	`"Lib01"`
Time and date	`"08:30, June 01, 2021"`

The Student ID is six characters long. The other two data items are of variable length.

A single string will be formed by concatenating the three data items. A separator character will need to be inserted between items two and three.

For example:

```
"CJL404Lib01<separator>08:30, June 01, 2021"
```

Each string represents one log entry.

A programmer decides to store the concatenated strings in a 1D array `LogArray` that contains 2000 elements. Unused array elements will contain an empty string.

(a) Suggest a suitable separator character **and** give a reason for your choice.

Character .....

Reason .....

.....

[2]

(c) A function `LogEvents()` will:

- take a Student ID as a parameter
- for each element in the array that matches the Student ID parameter:
  - add the value of the array element to the existing text file `LogFile`
  - assign an empty string to the array element
- count the number of lines added to the file
- return this count.

Write pseudocode for the function `LogEvents()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

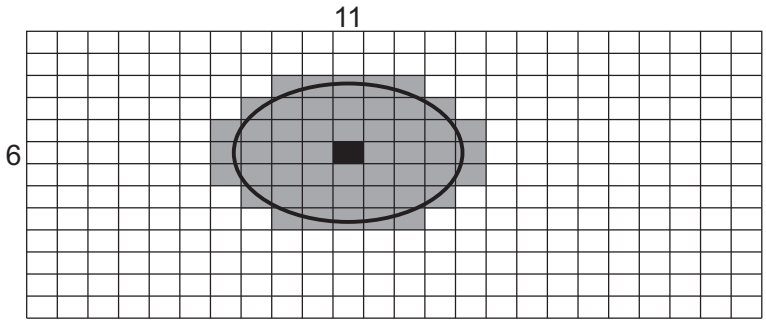
..... [7]

6 A mobile phone has a touchscreen. The screen is represented by a grid, divided into 800 rows and 1280 columns.

The grid is represented by a 2D array `Screen` of type `INTEGER`. An array element will be set to 0 unless the user touches that part of the screen.

Many array elements are set to 1 by a single touch of a finger or a stylus.

The following diagram shows a simplified touchscreen. The dark line represents a touch to the screen. All grid elements that are wholly or partly inside the outline will be set to 1. These elements are shaded.
The element shaded in black represents the centre point.



A program is needed to find the coordinates (the row and column) of the centre point. The centre point on the diagram above is row 6, column 11.

Assume:

- the user may only touch one area at a time
- screen rotation does not affect the touchscreen.

The programmer has started to define program modules as follows:

Module	Description
`SetRow()` (generates test data)	• Called with three parameters of type `INTEGER`: ◦ a row number ◦ the number of pixels to be skipped starting from column 1 ◦ the number of pixels that should be set to 1 • Sets the required number of pixels to 1 For example, `SetRow(3, 8, 5)` will give row 3 as in the diagram shown.
`SearchInRow()`	• Takes two parameters of type `INTEGER`: ◦ a row number ◦ a start column (1 or 1280) • Searches the given row from the start column (either left to right or right to left) for the first column that contains an element set to 1 • Returns the column number of the first element in the given row that is set to 1 • Returns −1 if no element is set to 1
`SearchInCol()`	• Takes two parameters of type `INTEGER`: ◦ a column number ◦ a start row (1 or 800) • Searches the given column from the start row (either up or down) for the first row that contains an element set to 1 • Returns the row number of the first element in the given column that is set to 1 • Returns −1 if no element is set to 1

(a) Write pseudocode to implement the module `SetRow()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [5]

(b) The module description of `SearchInRow()` is provided here for reference.

Module	Description
`SearchInRow()`	• Takes two parameters of type `INTEGER`: ◦ a row number ◦ a start column (1 or 1280) • Searches the given row from the start column (either left to right or right to left) for the first column that contains an element set to 1 • Returns the column number of the first element in the given row that is set to 1 • Returns −1 if no element is set to 1

Write pseudocode to implement the module `SearchInRow()`.

.....

.....

.....

.....

.....

.....

.....

..... [8]

Refer to the **insert** for the list of pseudocode functions and operators.

1 **(a)** A program is being developed to help manage the membership of a football club.

Complete the following identifier table.

Example value	Explanation	Variable name	Data type
`"Wong"`	The preferred name of the member joining the football club		
`FALSE`	A value to indicate whether an existing member of the club lives at the same address		
`19/02/1983`	When the member joined the football club		
`1345`	The number of points a member has earned. Members of the club earn points for different activities.		

[4]

(c) The following data items need to be stored for each student in a group:

- student name (a string)
- test score (an integer).

State a suitable data structure and justify your answer.

Structure .....

Justification .....

.....

.....

[3]

3 **(a)** A concert venue uses a program to calculate admission prices and store information about ticket sales.

A number of arrays are used to store data. The computer is switched off overnight and data has to be input again at the start of each day before any tickets can be sold. This process is very time consuming.

(i) Explain how the program could use text files to speed up the process.

.....

.....

.....

.....

.....

..... [2]

(ii) State the characteristic of text files that allow them to be used as explained in **part (a)(i)**.

.....

..... [1]

(iii) Information about ticket sales will be stored as a booking. The booking requires the following data:

- name of person booking
- number of people in the group (for example a family ticket or a school party)
- event type.

Suggest how data relating to each booking may be stored in a text file.

.....

.....

.....

..... [2]

(b) A procedure `Preview()` will:

- take the name of a text file as a parameter
- output a warning message if the file is empty
- otherwise output the first five lines from the file (or as many lines as there are in the file if this number is less than five).

Write pseudocode for the procedure `Preview()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [5]

5 A global 2D array `Result` of type `INTEGER` is used to store a list of exam candidate numbers together with their marks. The array contains 2000 elements, organised as 1000 rows and 2 columns.

Column 1 contains the candidate number and column 2 contains the mark for the corresponding candidate. All elements contain valid exam result data.

A procedure `Sort()` is needed to sort `Result` into ascending order of mark using an efficient bubble sort algorithm.

Write pseudocode for the procedure `Sort()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

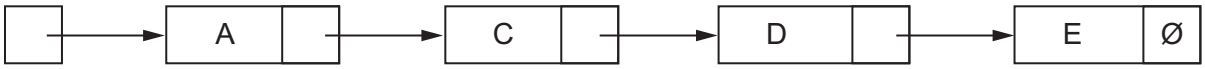
.....

.....

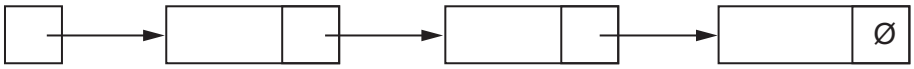
.....

..... [8]

6 The following diagram represents an Abstract Data Type (ADT) for a linked list.



The free list is as follows:



(a) Explain how a node containing data value B is added to the list in alphabetic sequence.

.....

.....

.....

.....

.....

.....

..... [4]

(b) Describe how the linked list in **part (a)** may be implemented using variables and arrays.

.....

.....

.....

..... [2]

7 A program is needed to take a string containing a full name and produce a new string of initials.

Some words in the full name will be ignored. For example, "the", "and", "of", "for" and "to" may all be ignored.

Each letter of the abbreviated string must be upper case.

For example:

Full name	Initials
Integrated Development Environment	IDE
The American Standard Code for Information Interchange	ASCII

The programmer has decided to use a global variable `FNString` of type `STRING` to store the full name.

It is assumed that:

- words in the full name string are separated by a single space character
- space characters will not occur at the beginning or the end of the full name string
- the full name string contains at least one word.

The programmer has started to define program modules as follows:

Module	Description
`GetStart()`	• Called with an `INTEGER` as a parameter, representing the number of a word in `FNString`. • Returns the character start position of that word in `FNString` or returns `-1` if that word does not exist • For example: if `FNString` contains the string `"hot and cold"`, `GetStart(3)` returns `9`
`GetWord()`	• Called with a parameter representing the position of the first character of a word in `FNString` • Returns the word from `FNString` • For example: if `FNString` contains the string `"hot and cold"`, `GetWord(9)` returns `"cold"`

(a) Write pseudocode for the module `GetStart()`.

.....

.....

.....

.....

.....

.....

..... [7]

(b) The programmer has decided to use a global ten-element 1D array `IgnoreList` of type `STRING` to store the ignored words. Unused elements contain the empty string (`""`) and may occur anywhere in the array.

A new module `AddWord()` is needed as follows:

Module	Description
`AddWord()`	• Called with a parameter representing a word • Stores the word in an unused element of the `IgnoreList` array and returns `TRUE` • Returns `FALSE` if the array was already full or if the word was already in the array

Write a detailed description of the algorithm for `AddWord()`. Do **not** include pseudocode statements in your answer.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [4]

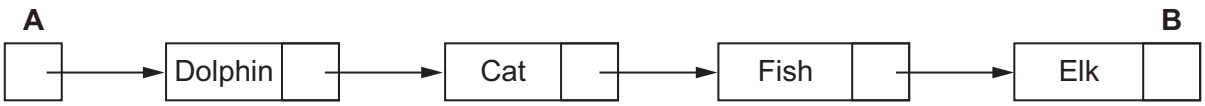
Refer to the **insert** for the list of pseudocode functions and operators.

1 (a) (i) Complete the following table by giving the appropriate data type in each case.

Variable	Example data value	Data type
`Name`	`"Catherine"`	
`Index`	`100`	
`Modified`	`FALSE`	
`Holiday`	`25/12/2020`	

[4]

3 The following diagram represents an Abstract Data Type (ADT).



(a) Identify this type of ADT.

..... [1]

(b) Give the technical term for the item labelled **A** in the diagram.

..... [1]

(c) Give the technical term for the item labelled **B** in the diagram.

Explain the meaning of the value given to this item.

Term .....

Meaning .....

.....

.....

[2]

(d) Complete the diagram to show the ADT after the data has been sorted in alphabetical order.



[2]

5 **(a)** A student is learning about arrays.

She wants to write a program to:

- declare a 1D array `RNum` of 100 elements of type `INTEGER`
- assign each element a random value in the range 1 to 200 inclusive
- count and output how many numbers generated were between 66 and 173 inclusive.

(i) Write pseudocode to represent the algorithm.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [6]

(ii) The student decides to modify the algorithm so that each element of the array will contain a unique value.

Describe the changes that the student needs to make to the algorithm.

.....

.....

.....

.....

.....

..... [3]

6 A procedure `CountVowels()` will:

- be called with a string containing alphanumeric characters as its parameter
- count and output the number of occurrences of each vowel (a, e, i, o, u) in the string
- count and output the number of occurrences of the other alphabetic characters (as a single total).

The string may contain both upper and lower case characters.

Each count value will be stored in a unique element of a global 1D array `CharCount` of type `INTEGER`. The array will contain six elements.

Write pseudocode for the procedure `CountVowels()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [8]

8 A program is needed to take a string containing a full name and to produce a new string of initials.

Some words in the full name will be ignored. For example, “the”, “and”, “of”, “for” and “to” may all be ignored.

Each letter of the new string must be upper case.

For example:

Full name	Initials
Integrated Development Environment	IDE
The American Standard Code for Information Interchange	ASCII

The programmer has decided to use the following global variables:

- a ten element 1D array `IgnoreList` of type `STRING` to store the ignored words
- a string `FNString` to store the full name string.

Assume that:

- each alphabetic character in the full name string may be either upper or lower case
- the full name string contains at least one word.

The programmer has started to define program modules as follows:

Module	Description
`GetStart()`	• Called with an `INTEGER` as its parameter, representing the number of a word in `FNString` • Returns the character start position of that word in `FNString` or returns `-1` if that word does not exist • For example: `GetStart(3)` applied to `"hot and cold"` returns `9`
`GetWord()`	• Called with the position of the first character of a word in `FNString` as its parameter • Returns the word from `FNString` • For example: if `FNString` contains the string `"hot and cold"`, `GetWord(9)` returns `"cold"`
`IgnoreWord()`	• Called with a `STRING` parameter representing a word • Searches for the word in the `IgnoreList` array • Returns `TRUE` if the word is found, otherwise returns `FALSE`
`GetInitials()`	• Processes the sequence of words in the full name one word at a time • Calls `GetStart()`, `GetWord()` and `IgnoreWord()` to process each word to form the new string • Outputs the new string

(a) Write pseudocode for the module `IgnoreWord()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [5]

Refer to the **insert** for the list of pseudocode functions and operators.

1 **(a)** A program is being developed to help manage the membership of a football club.

Complete the following identifier table.

Example value	Explanation	Variable name	Data type
`"Wong"`	The preferred name of the member joining the football club		
`FALSE`	A value to indicate whether an existing member of the club lives at the same address		
`19/02/1983`	When the member joined the football club		
`1345`	The number of points a member has earned. Members of the club earn points for different activities.		

[4]

(c) The following data items need to be stored for each student in a group:

- student name (a string)
- test score (an integer).

State a suitable data structure and justify your answer.

Structure .....

Justification .....

.....

.....

[3]

3 **(a)** A concert venue uses a program to calculate admission prices and store information about ticket sales.

A number of arrays are used to store data. The computer is switched off overnight and data has to be input again at the start of each day before any tickets can be sold. This process is very time consuming.

(i) Explain how the program could use text files to speed up the process.

.....

.....

.....

.....

.....

..... [2]

(ii) State the characteristic of text files that allow them to be used as explained in **part (a)(i)**.

.....

..... [1]

(iii) Information about ticket sales will be stored as a booking. The booking requires the following data:

- name of person booking
- number of people in the group (for example a family ticket or a school party)
- event type.

Suggest how data relating to each booking may be stored in a text file.

.....

.....

.....

..... [2]

(b) A procedure `Preview()` will:

- take the name of a text file as a parameter
- output a warning message if the file is empty
- otherwise output the first five lines from the file (or as many lines as there are in the file if this number is less than five).

Write pseudocode for the procedure `Preview()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [5]

5 A global 2D array `Result` of type `INTEGER` is used to store a list of exam candidate numbers together with their marks. The array contains 2000 elements, organised as 1000 rows and 2 columns.

Column 1 contains the candidate number and column 2 contains the mark for the corresponding candidate. All elements contain valid exam result data.

A procedure `Sort()` is needed to sort `Result` into ascending order of mark using an efficient bubble sort algorithm.

Write pseudocode for the procedure `Sort()`.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

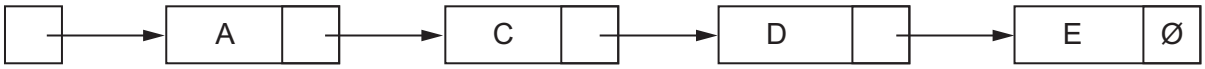
.....

.....

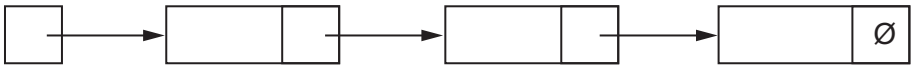
.....

..... [8]

6 The following diagram represents an Abstract Data Type (ADT) for a linked list.



The free list is as follows:



(a) Explain how a node containing data value B is added to the list in alphabetic sequence.

.....

.....

.....

.....

.....

.....

..... [4]

(b) Describe how the linked list in **part (a)** may be implemented using variables and arrays.

.....

.....

.....

..... [2]

7 A program is needed to take a string containing a full name and produce a new string of initials.

Some words in the full name will be ignored. For example, "the", "and", "of", "for" and "to" may all be ignored.

Each letter of the abbreviated string must be upper case.

For example:

Full name	Initials
Integrated Development Environment	IDE
The American Standard Code for Information Interchange	ASCII

The programmer has decided to use a global variable `FNString` of type `STRING` to store the full name.

It is assumed that:

- words in the full name string are separated by a single space character
- space characters will not occur at the beginning or the end of the full name string
- the full name string contains at least one word.

The programmer has started to define program modules as follows:

Module	Description
`GetStart()`	• Called with an `INTEGER` as a parameter, representing the number of a word in `FNString`. • Returns the character start position of that word in `FNString` or returns `-1` if that word does not exist • For example: if `FNString` contains the string `"hot and cold"`, `GetStart(3)` returns `9`
`GetWord()`	• Called with a parameter representing the position of the first character of a word in `FNString` • Returns the word from `FNString` • For example: if `FNString` contains the string `"hot and cold"`, `GetWord(9)` returns `"cold"`

(a) Write pseudocode for the module `GetStart()`.

.....

.....

.....

.....

.....

.....

..... [7]

(b) The programmer has decided to use a global ten-element 1D array `IgnoreList` of type `STRING` to store the ignored words. Unused elements contain the empty string (`""`) and may occur anywhere in the array.

A new module `AddWord()` is needed as follows:

Module	Description
`AddWord()`	• Called with a parameter representing a word • Stores the word in an unused element of the `IgnoreList` array and returns `TRUE` • Returns `FALSE` if the array was already full or if the word was already in the array

Write a detailed description of the algorithm for `AddWord()`. Do **not** include pseudocode statements in your answer.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

..... [4]